MITRE ATT&CK-Based Mini SOC: Attack Simulation, Detection & DFIR

Case Creation

Ravintharan Sanjiv

Aspiring Cybersecurity Analyst

July 2026

All attacks were conducted within a controlled laboratory environment for education and authorized testing purposes only.

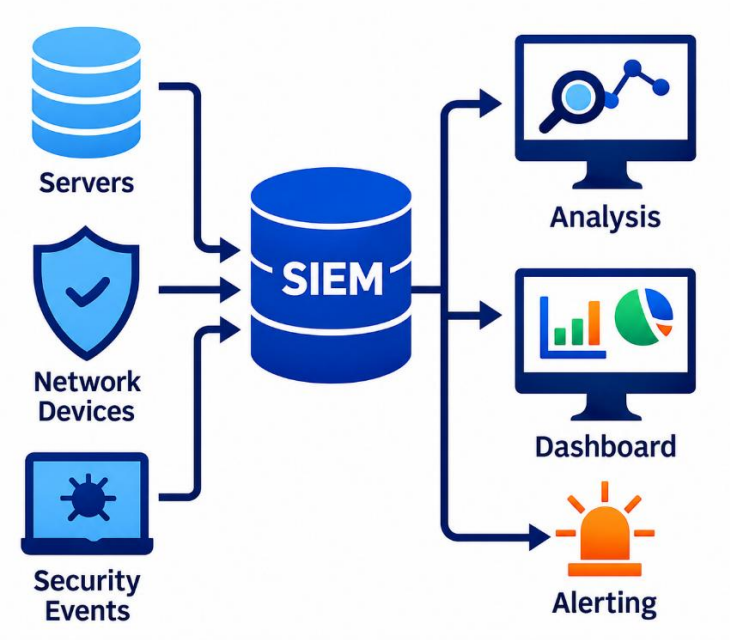
Abstract

In this project I have installed a security information and event management system (SIEM) named wazuh and integrated Sysmon as the endpoint telemetry tool to give wazuh more detailed events and also syscheck to configure file integrity monitoring feature in wazuh. Then wazuh agent was used to provision a windows endpoint, together they are used to analyze events and generate alerts to the simulated attacks in the project namely Nmap scan, brute forcing using hydra, PowerShell reverse shell (remote code execution) and file integrity violation. These attacks were mapped to MITRE ATT&CK framework to understand the attack lifecycle and standardize the adversary behavior. Some attacks didn't generate any alert since it can cause alert fatigue to the analyst for how insignificant the attack is in an enterprise level, but for the purpose of this project, I wrote custom detection rules to generate alerts regardless. At last, using DFIR-IRIS case management system, cases were created and assigned to mock soc analysts to imitate a real world security operation center. This project was created to bridge the gap between theoretical cybersecurity knowledge and practical experience. By simulating a real cyber attack detection, analysis and investigation lifecycle.

Table of Content

Table of Figures

Introduction



Modern businesses are struggling to detect cyber attacks because each system must be checked separately to see a full attack since these events are individual and looks independent inducing a major attack could go unnoticed without centralized correlation of events. The enormous number of logs generated can also be time consuming and exhausting to analyze making all these problems contribute to high risk of missing out real attacks. This is why a SIEM (Security Information Management System) is used and for this project I have implemented wazuh a open source security monitoring tool that provides capabilities associated with SIEM and XDR(Extended Detection and Response) designed to collect, analyze and monitor endpoints, servers, network devices and other infrastructure components for suspicious activity. But it is also important to manage the security events among the security teams, investigate and document the incidents and for this purpose DFIR-IRIS acts as the centralized workplace platform.

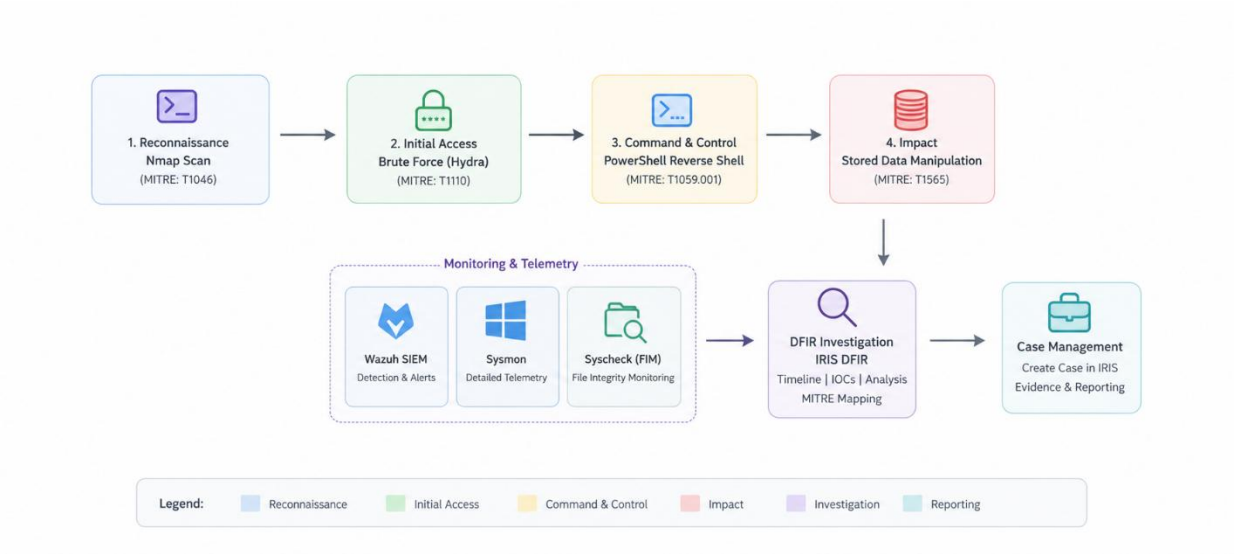
However, for aspiring security professional learning concepts of log analysis, threat detection, incident response and malware analysis can only get them so far and its challenging to get the opportunity or the access to learn in realistic SOC environments. For this reason I have built a mini SOC laboratory environment that consist of real world attacks and investigations using industry relevant security tools such as wazuh and IRIS to gain a practical SOC experience.

In this project I have simulated MITRE ATT&CK mapped attacks like Nmap scan, brute forcing, PowerShell reverse shell and stored data manipulation. Then these attacks were monitored using wazuh and alerts were generated according to the custom detection rules. Lastly cases were created for each event using IRIS for centralized management and investigation.

Project Objective

- Deploying Wazuh management stack on the server
- Installing DFIR-IRIS case management system
- Registering a host using Wazuh agent
- Sysmon and syscheck integration with Wazuh
- Writing custom detection rules for wazuh
- Simulating attacks and mapping them to MITRE ATT&CK framework
- Detecting the simulated attacks using Wazuh
- Case creation in IRIS

Flow Diagram and System Environment



Role	Operating system	Hypervisor	Network Adapter
Host Machine	Windows 11 Home 25H2	N/A	Bridged Network
Victim Machine	Windows 10 Home 21H2	VMware Workstation 17 Pro 17.5.2	
Attacker Machine	Kali GNU/Linux Rolling 2025.4		
Hosting Wazuh SIEM	Ubuntu 22.04.5 LTS		
Hosting Iris DFIR Platform			

Environment Setup

1. Wazuh Installation

1.1 Authenticating Wazuh Package

curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg -- dearmor -o /usr/share/keyrings/wazuh-archive-keyring.gpg

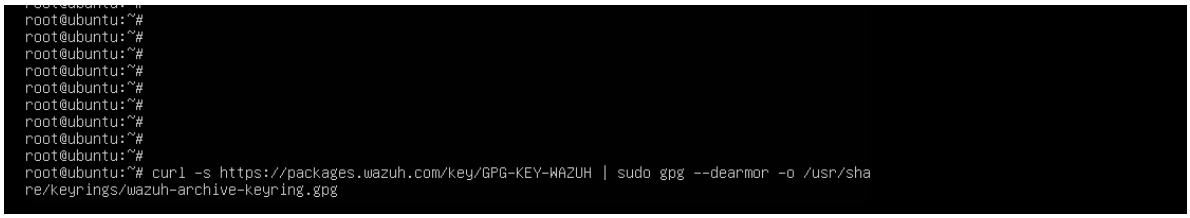


Figure 1: Verifying the authenticity of Wazuh packages

The first half of the command downloads the Wazuh's public key from the official wazuh package server and the second half converts the ASCII armored public key into binary format, which the APT can use later. This process is done to verify whether the package we are downloading is authenticate and not tampered in between or before the download.

1.2 Installing Wazuh SIEM

curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a

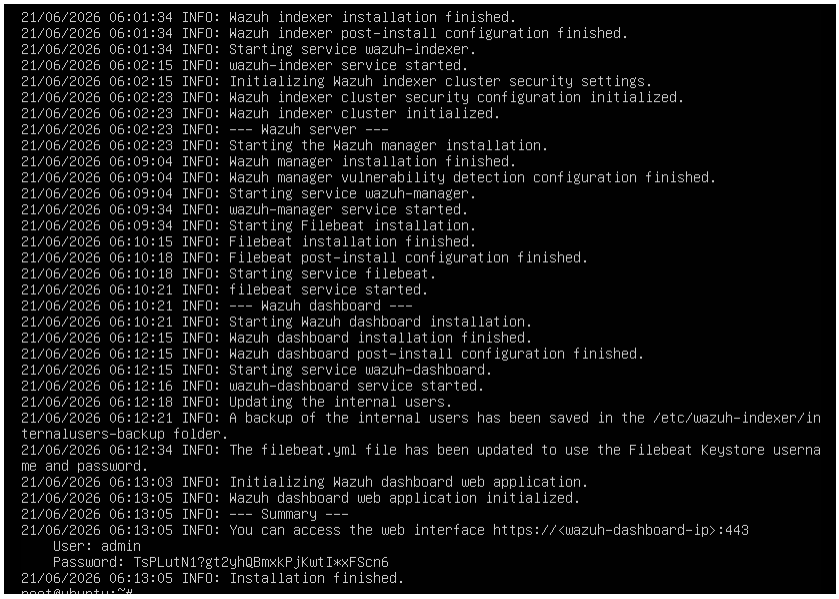


Figure 2: Quick Installation of Wazuh

Instead of manually installing wazuh components, I used the Quick Installation command. After successfully installing the package, I got my admin account password and the port number that is used to access the SIEM.

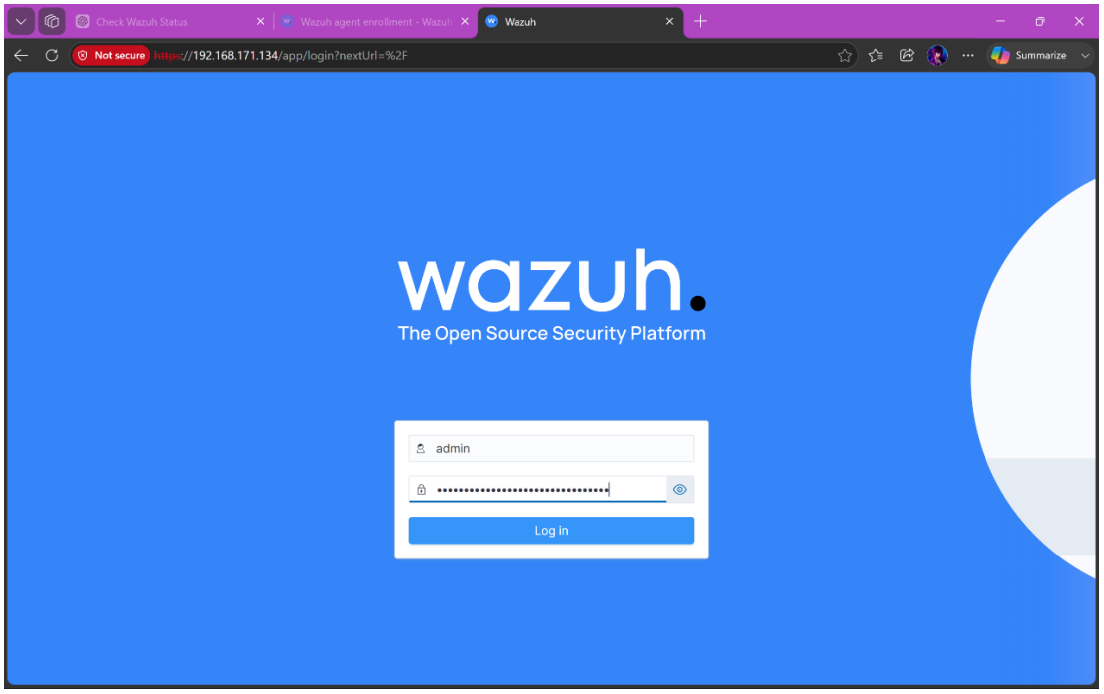


Figure 3: Wazuh login page

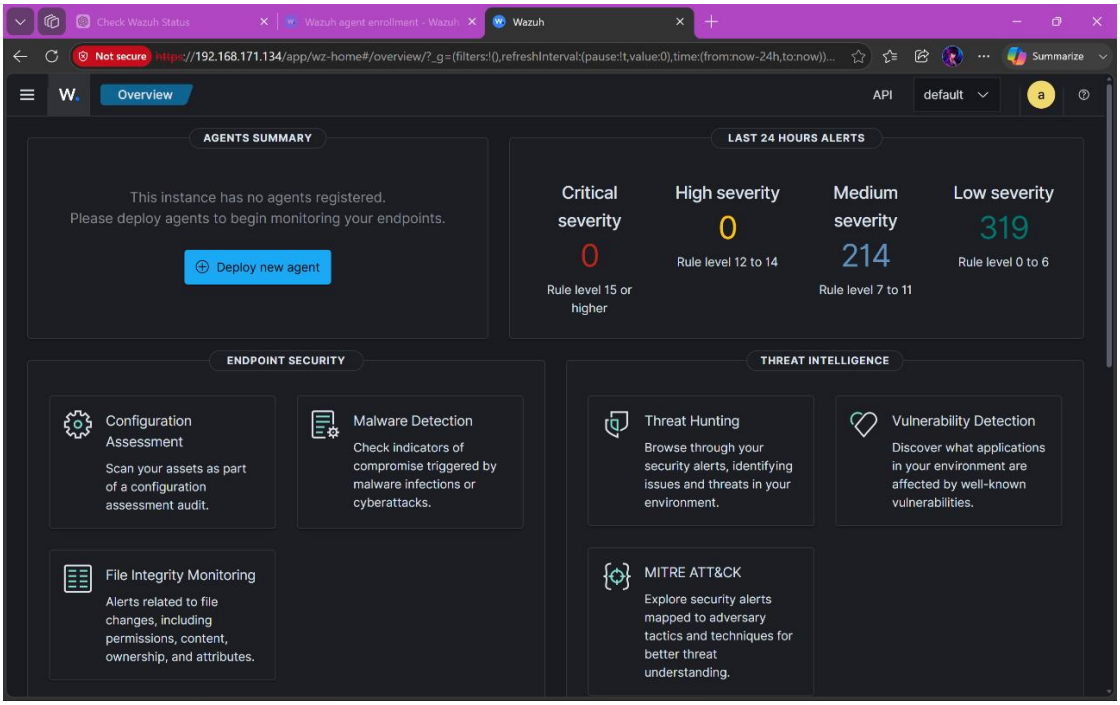


Figure 4: Wazuh dashboard

Successfully installed Wazuh SIEM and enabled dark mode. The SIEM was then accessed from the Windows host machine using the Ubuntu sever's IP address.

1.3 Adding Detection Rules

"/var/ossec/etc/rules/local_rules.xml

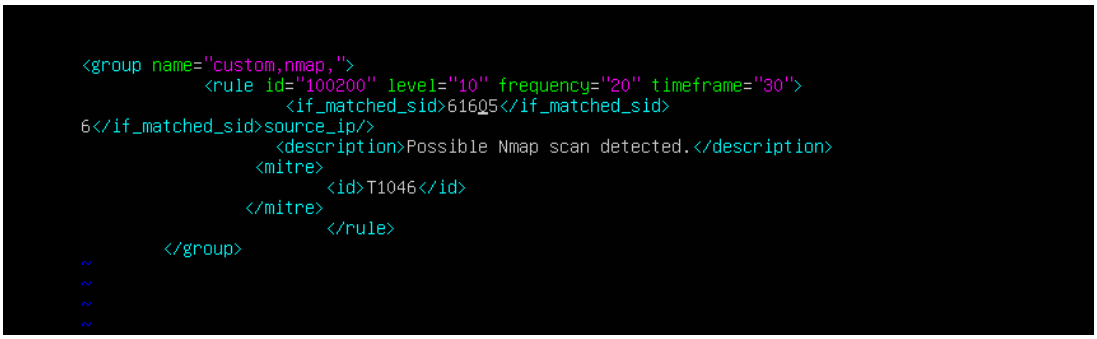


Figure 5: Detection rule for Nmap scans

Since wazuh isnt a network Intrusion detectection system in default it is doesn't log network activities, therefore I made the wazuh agent in the windows to collect the sysmon event logs and created a custom rule to generate an alert if the same source ip generates 20 sysmon network connection events within 30 seconds .

Figure 6: Detection rule for ssh brute force attack

This is a two stage detection and correlation rule, the first rule detects a single failed login, and then the second rule detects the brute force.This custome rule gets triggered if the 1st rule gets triggeed 5 times within 60 seconds from the same ip and tags it to the MITRE ATT&CK mapping T110.

Figure 7: Detection rule validation and service restart

sudo /var/ossec/bin/wazuh-analysisd -t

systemct1 restart wazuh-manager.service

The added Detection rules are then validated using the above command and then the wazuh service is restarted.

2. Iris Installation

docker-compose – version

git clone https://github.com/dfir-iris/iris-web.git

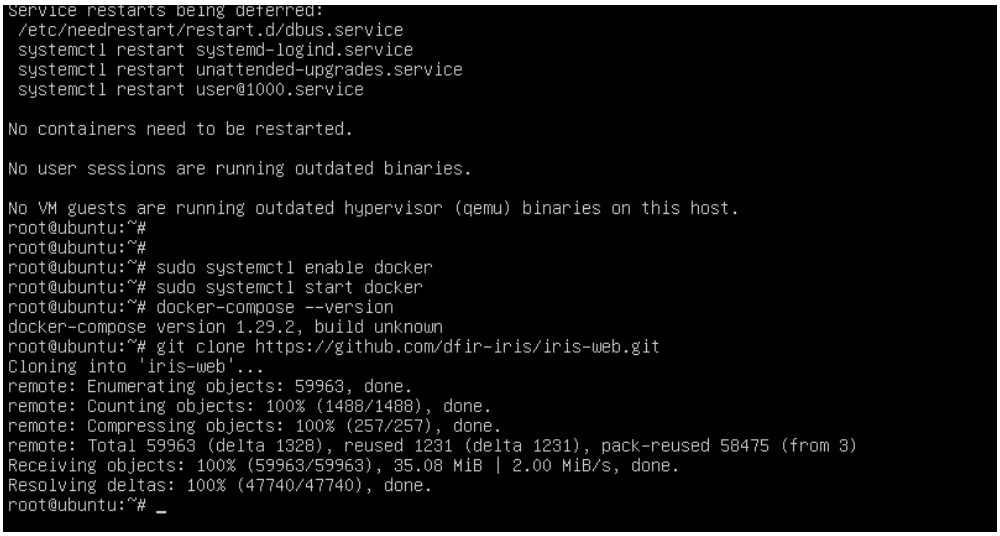


Figure 8: Clonning Iris from github

I previously installed docker since Iris is a containerized application and then I cloned the Iris Project from the gitub , this contains the blueprints of how to build the Iris webapp, it consists of wich container to create, what ports to expose, which image the container should use and etc.

docker compose pull

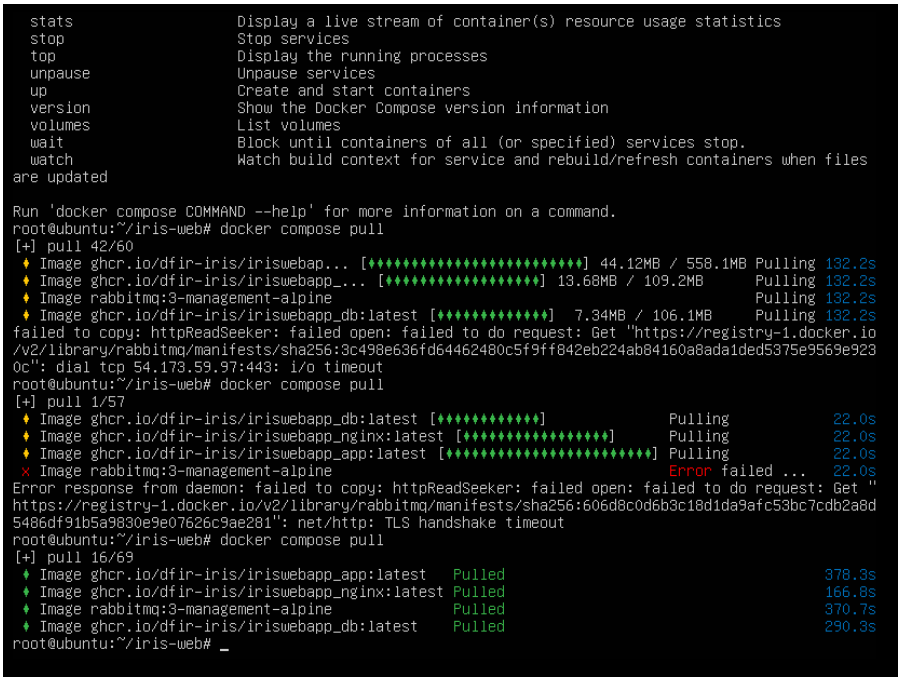


Figure 9: Downloading Iris components

Now the docker opens the docker-compose.yml file, which was installed previously from github. Now that it knows the instructions of like, what image to use and how containers should communicate, docker proceeds with installing them from docker hub or github container registry.

docker compose up

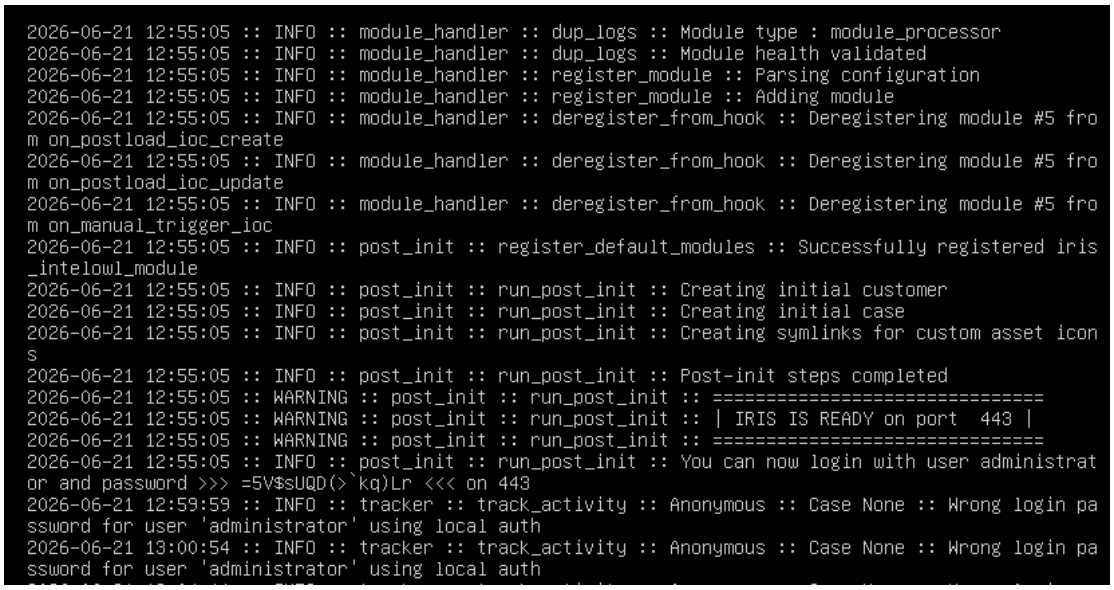


Figure 10: Installing Iris

Now the docker starts assembling the images and other components and starts running containers. Once its finished it states the port number the tool should be accessed and the password for admin access.

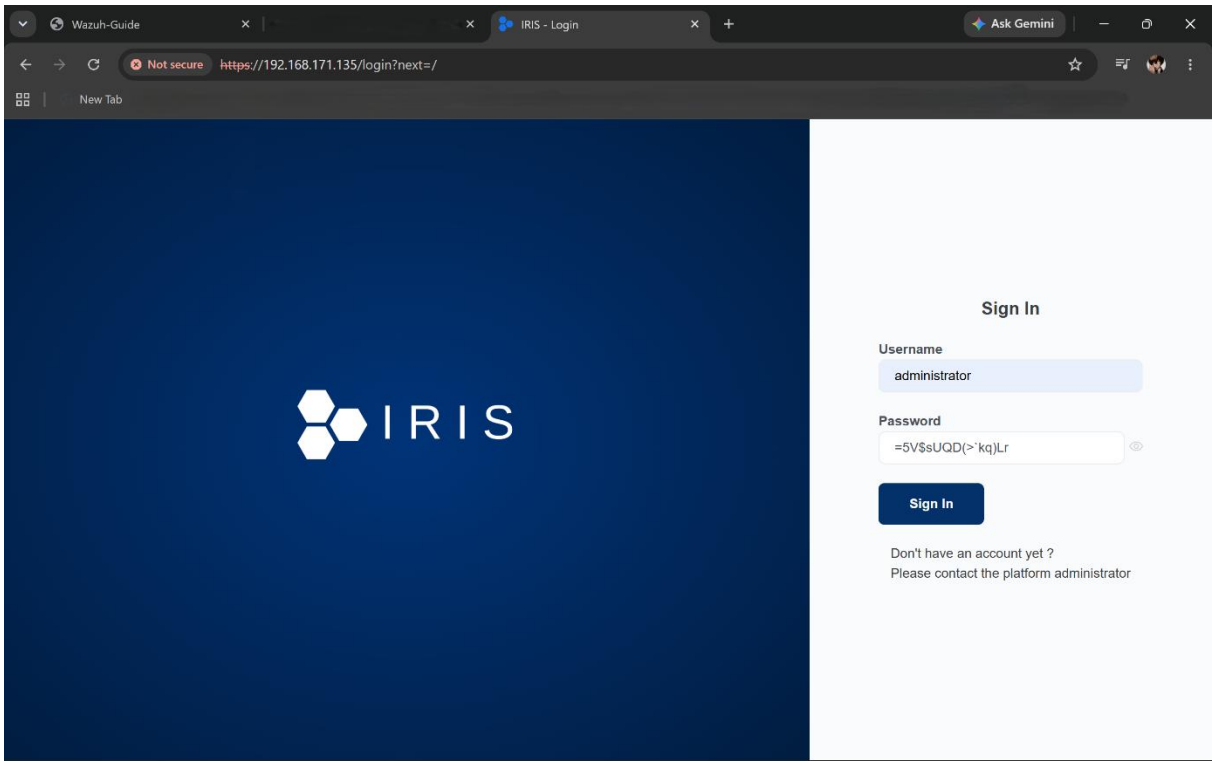


Figure 11: Iris login page

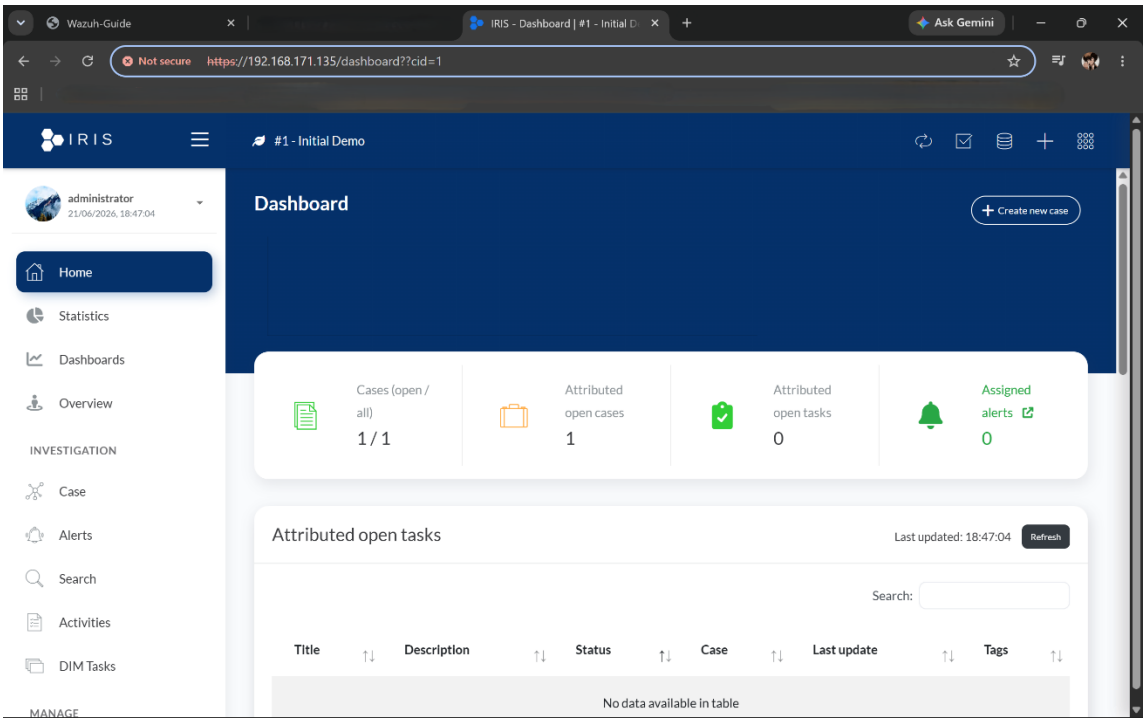


Figure 12: Iris dashboard

3. Registering the Windows machine as a Wazuh agent

3.1 Deploying Windows host as a Wazuh agent

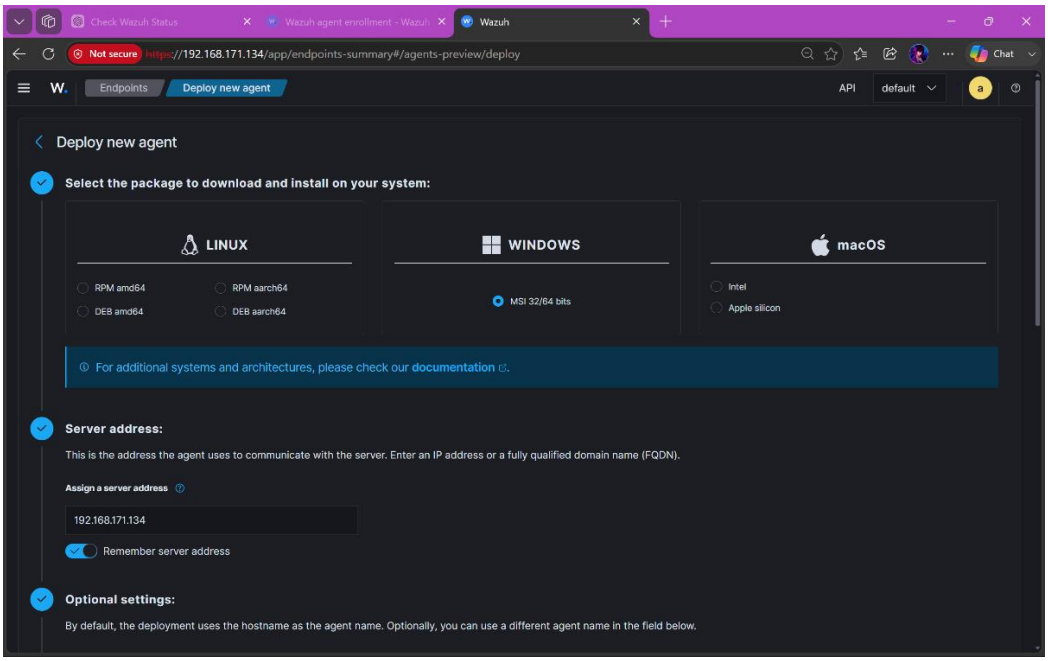


Figure 13: Adding Windows as an agent in Wazuh (1)

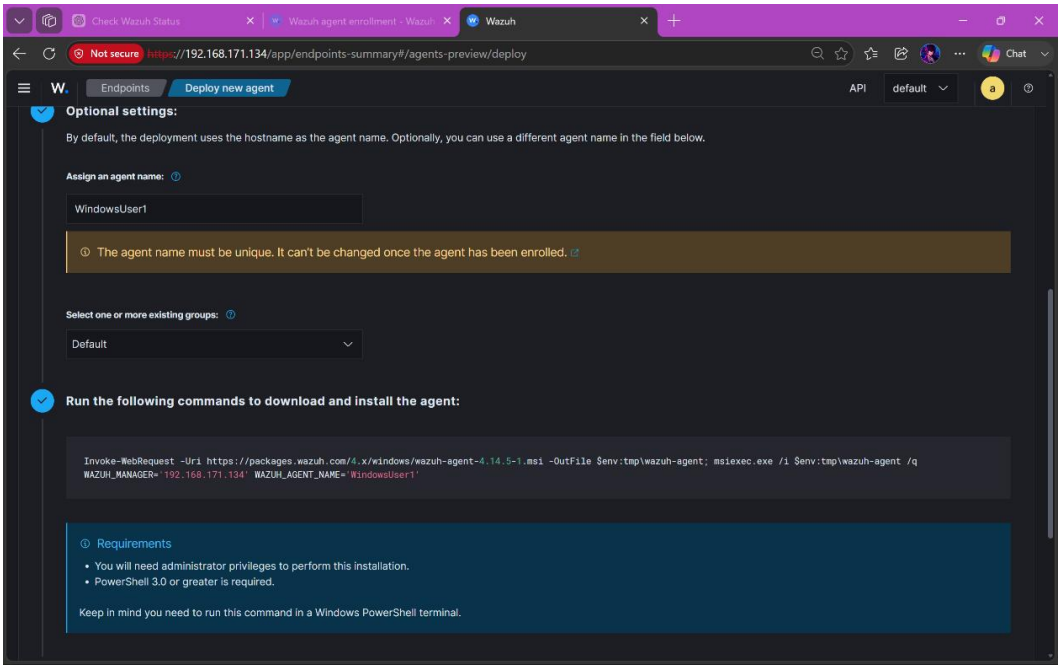


Figure 14: Adding Windows as an agent in Wazuh (2)

Here I deployed configured it by assigning the Wazuh's IP as the server address and agent name as WindowsUser1.

NET START Wazuh

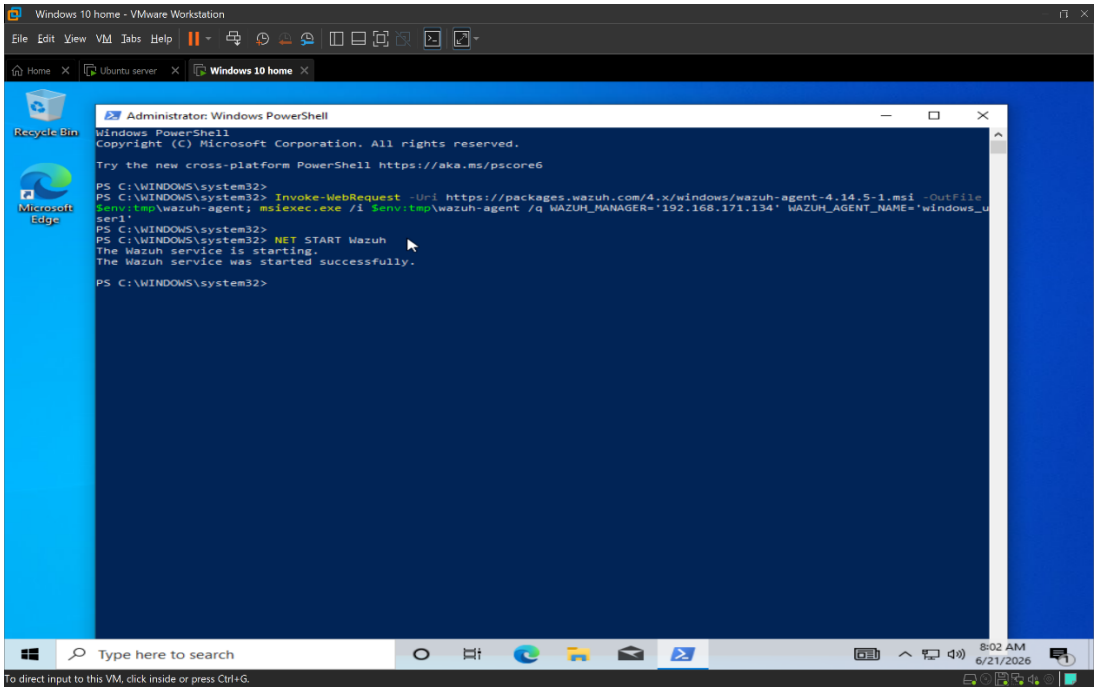


Figure 15: Configuring Windows as an Wazuh agent in Windows

After pasting the command that was given by the wazuh, I started the Wazuh service.

3.2 Confirming Windows agent Deployment

sudo /var/ossec/bin/agent_control -l

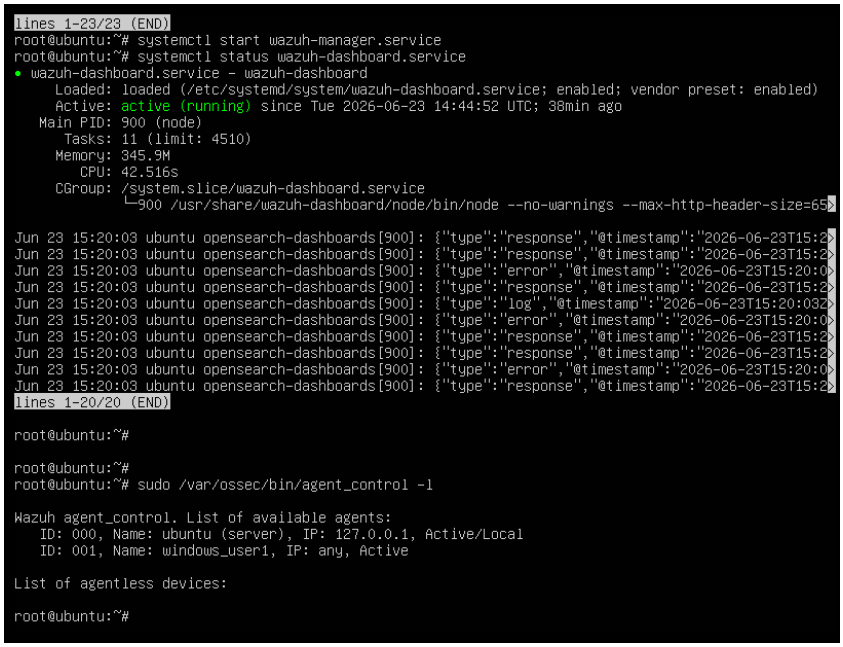


Figure 16: Confirming the agent deployment from the Wazuh server

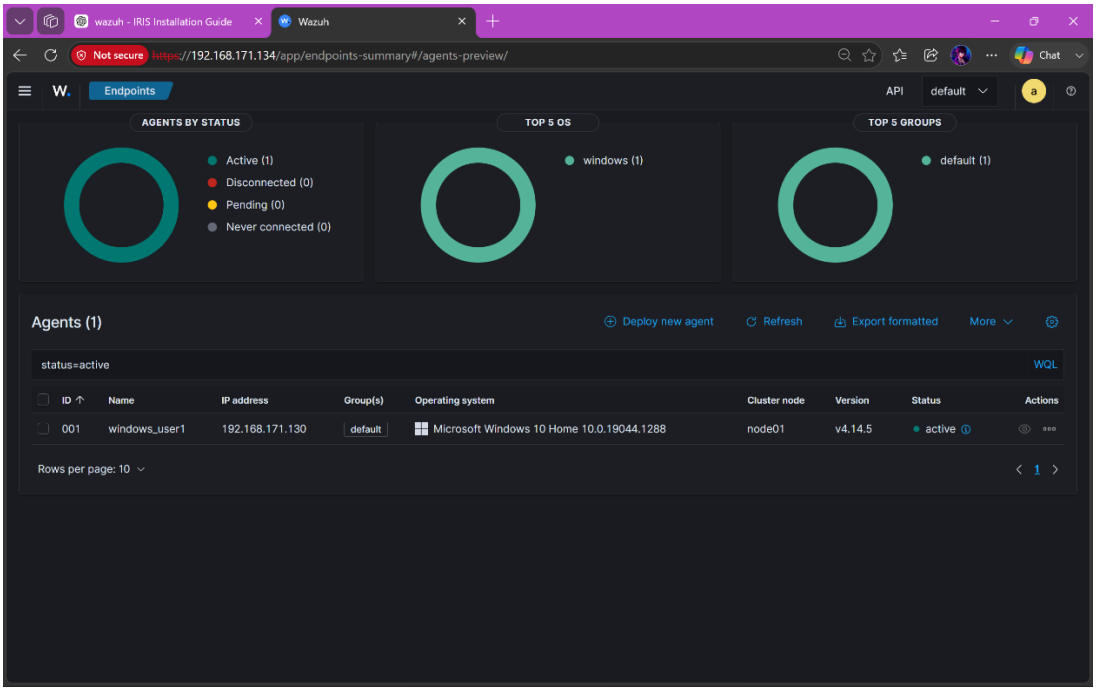


Figure 17: Confirming the agent deployment from the Wazuh webapp

This confirms that, we have successfully deployed the Windows agent. In the Wazuh webapp, the windows ip address, Windows version and the status is visible.

4. Sysmon Installation

4.1 Installing and Configuring Sysmon

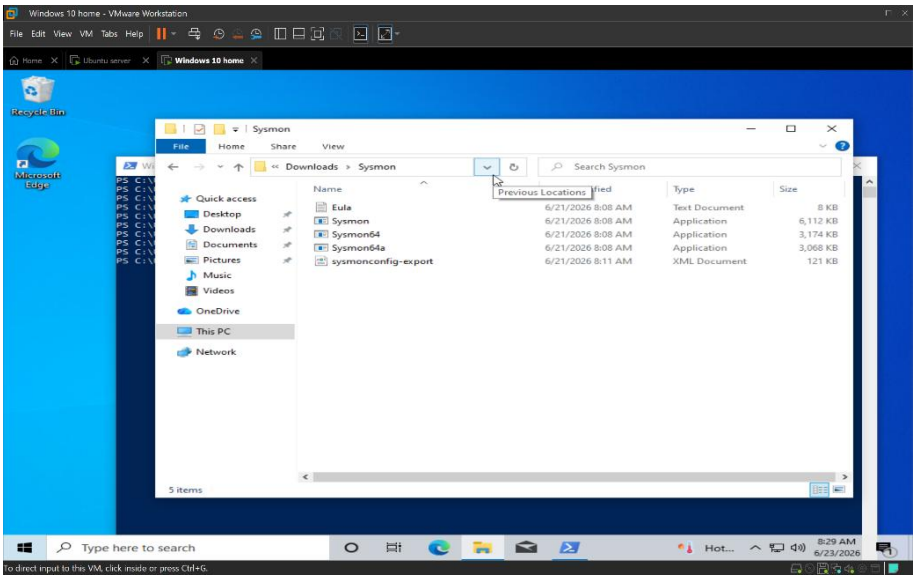


Figure 18: Preinstalled Sysmon files

.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml

Get-service Sysmon64

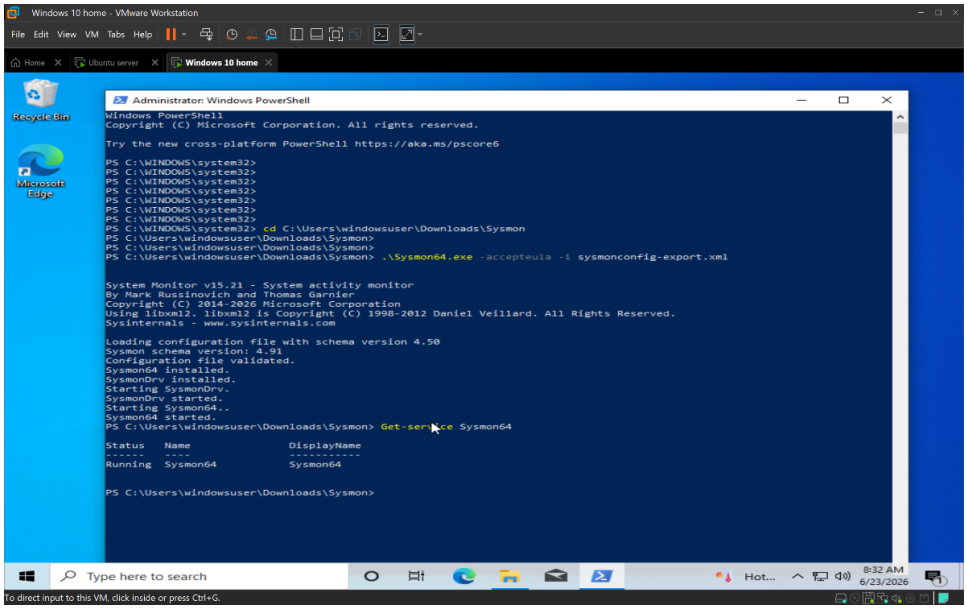


Figure 19: Installation and configuration of Sysmon

This command installs Sysmon as a Windows service and loads the sysmonconfig-export.xml configuration file that has the instructions on what the Sysmon has to monitor and then the second command is used to check the status of the service.

4.2 Forwarding Sysmon Logs to Wazuh

C:\Program Files (x86)\ossec-agent\ossec.conf

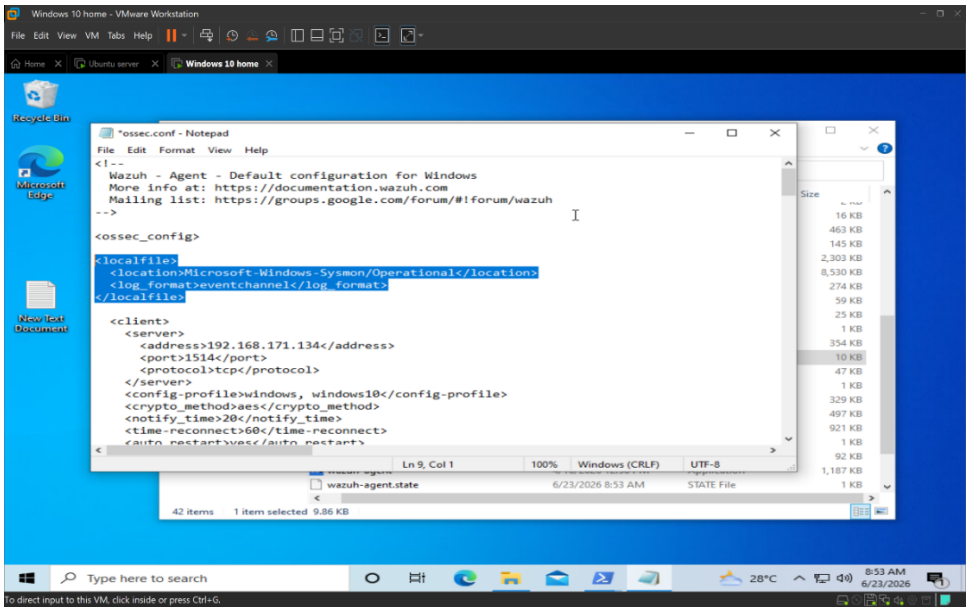


Figure 20: Forwarding Sysmon logs to Wazuh manager

This edit in the ossec.conf file is done to forward Sysmon logs from the Windows event log to Wazuh manager for analysis. Basically the wazuh agent says to monitor Sysmon operational event log which conatains process creation, Registry changes and etc. It also specifies the source of Windows event channel so Wazuh knows to use the Windows event log API to extract information instead of reading a text log file.

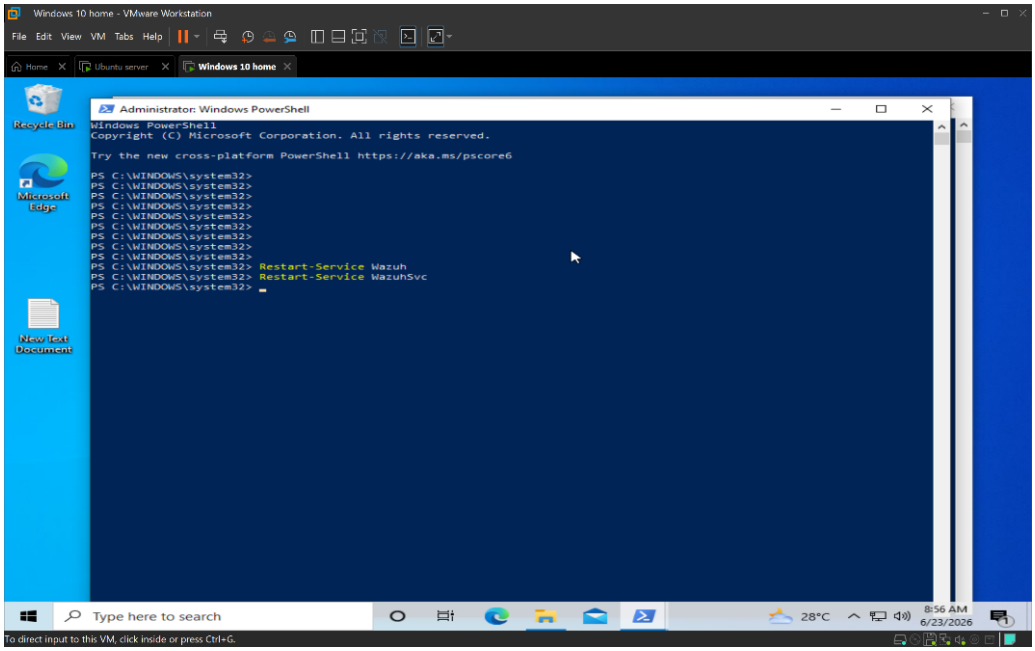


Figure 21: Restarting the Wazuh service to save changes

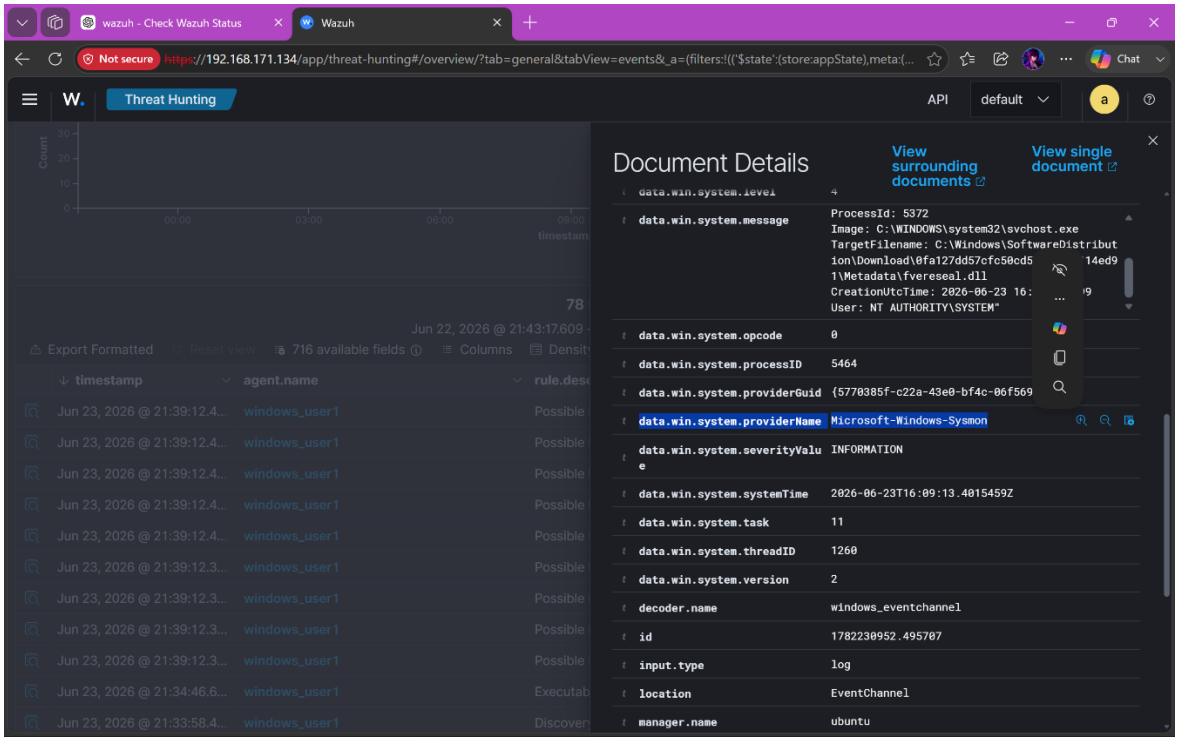


Figure 22: Confirming wazuh recieves the sysmon logs

5. Configuring Syscheck

C:\User1\Sensitive Data

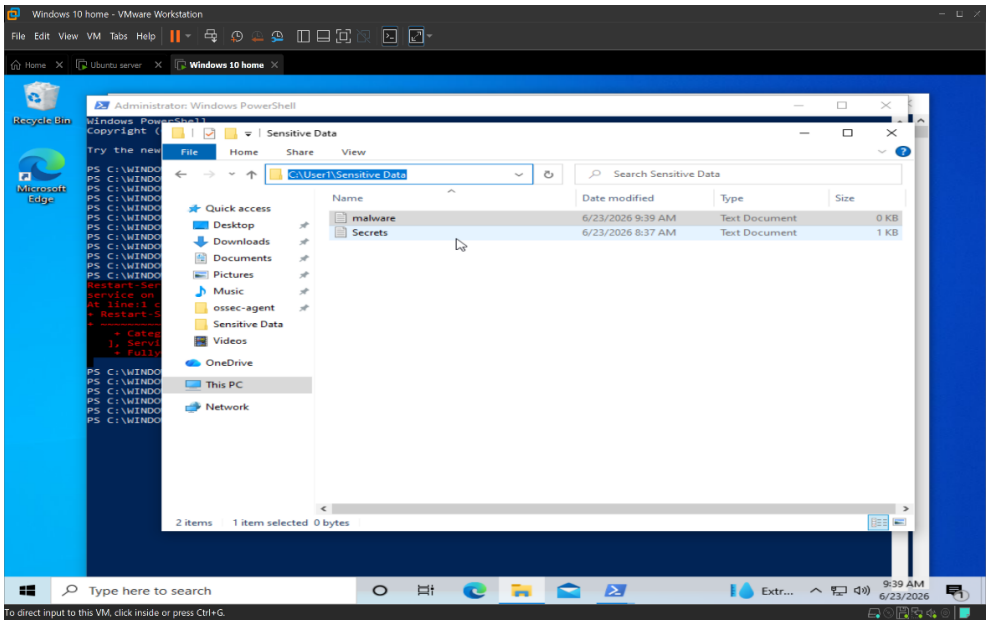


Figure 23: The folder that will be monitored

<directories realtime="yes">C: \User1\Sensitive Data</directories>

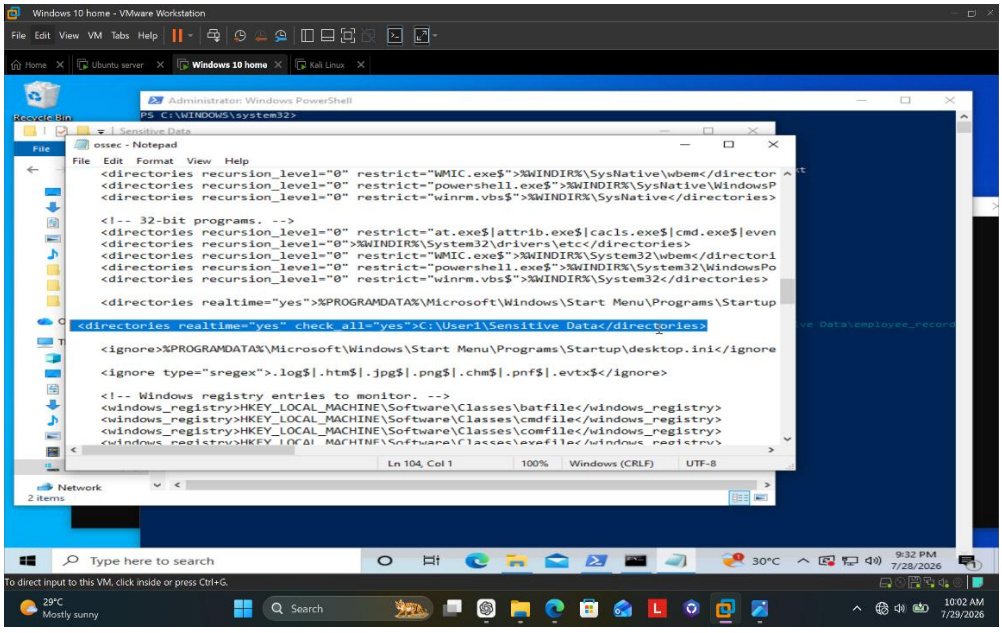


Figure 24: Establishing the folder to be monitored

The above line, which has the folder path to be monitored is added under the file integrity monitoring section inside the same C:\Program Files (x86)\ossec-agent\ossec.conf file.

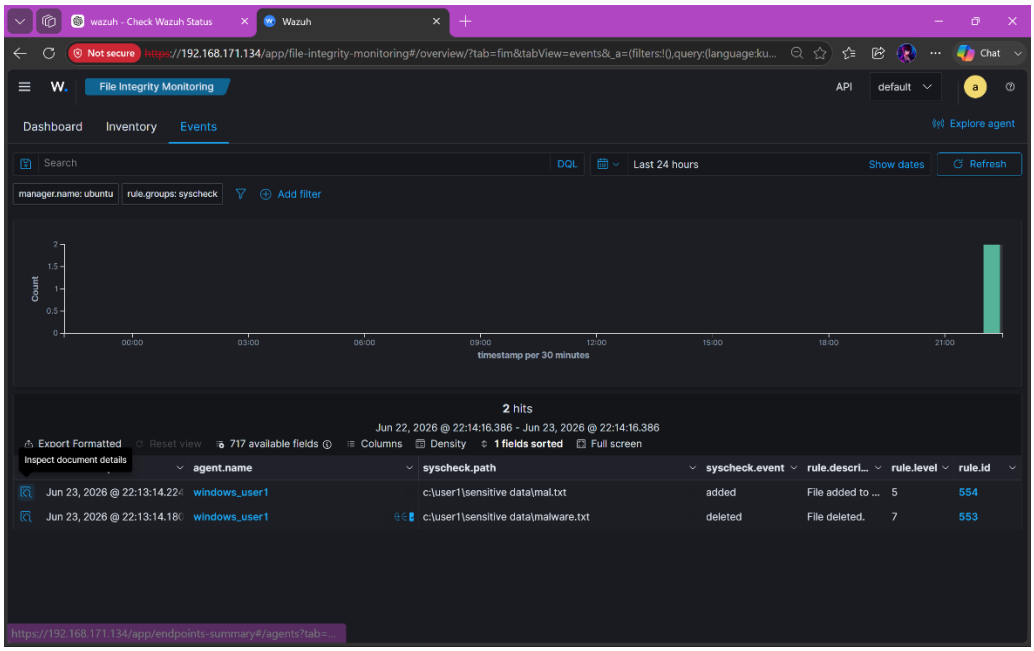


Figure 25: Confirmation of wazuh FIM

6. Making the Windows machine Vulnerable

6.1 Enabling SSH service

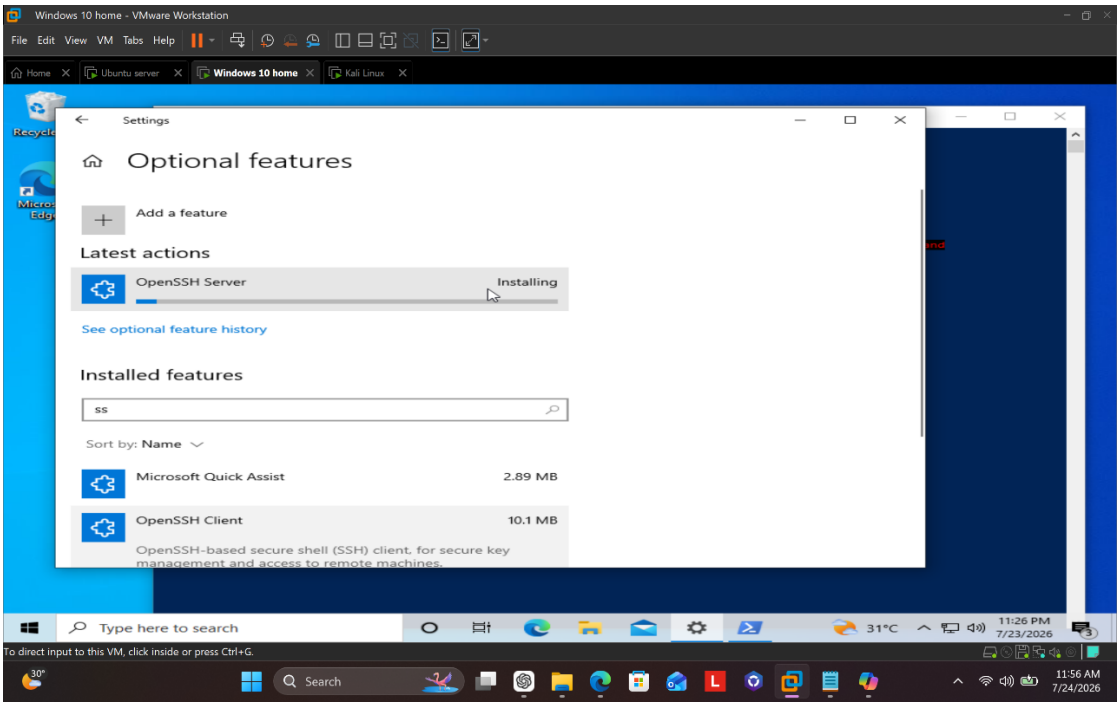


Figure 26: Installing SSH

START-Service sshd

GET-SERVICE sshd

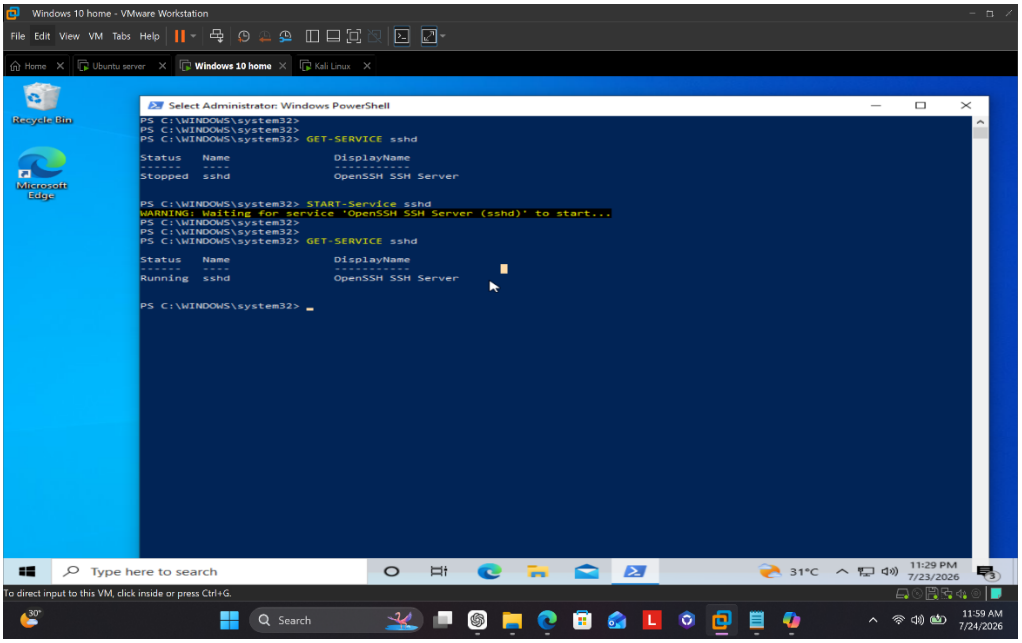


Figure 27: Starting the SSH service

Since Windows use RDP(Remote Desktop Protocol) for remote administration, I had to install SSH as an optional feature, and made sure the services are running using above powershell commands.

6.2 Disabling Windows Security features

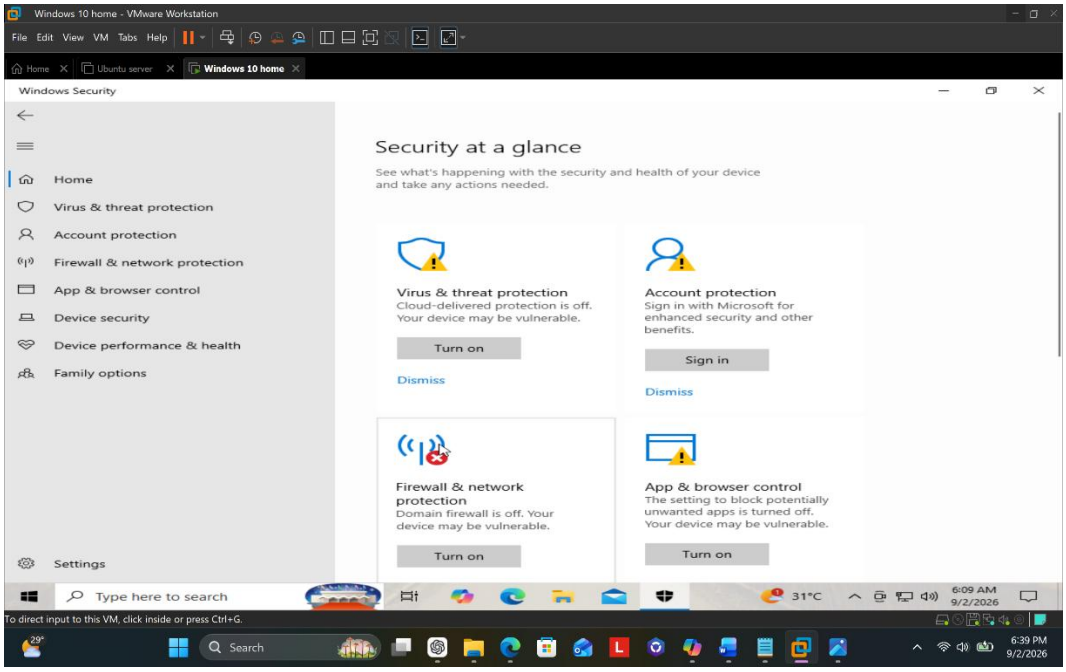


Figure 28: Disabling Windows default Security features

Attack Simulation and Detection

1. Network Enumeration using Nmap (T1046)

1.1 Performing Reconnaissance [TA0043]

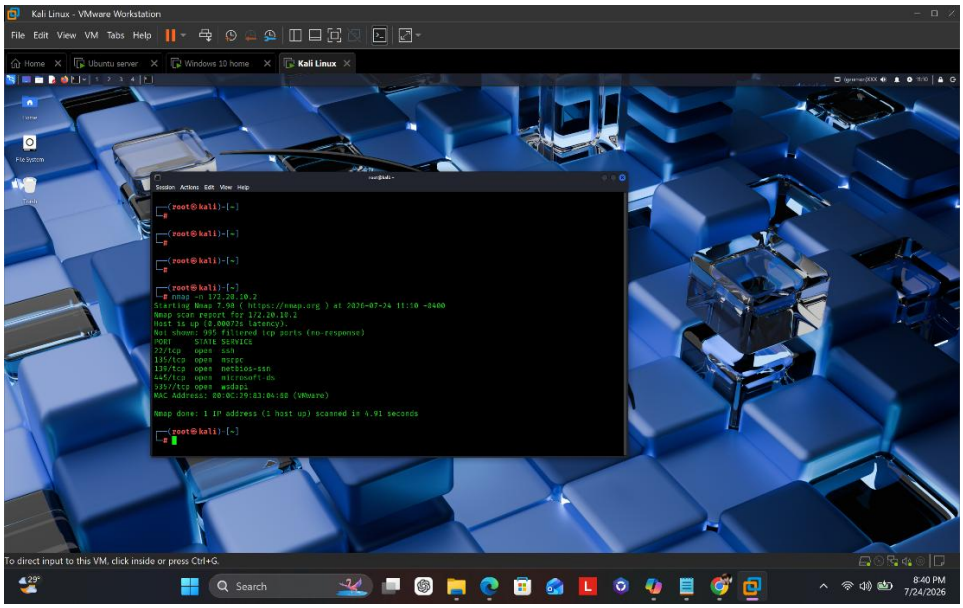


Figure 29: Nmaping to windows machine

Scanned the machines's ip to find open services and as intended ssh port is open.

1.2 Wazuh Detection

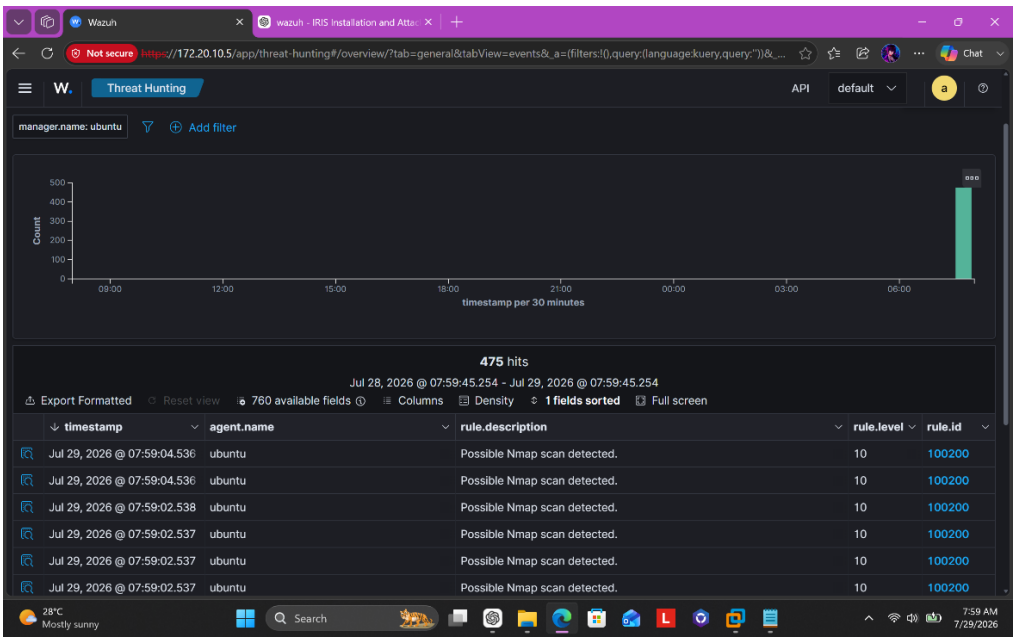


Figure 30: Alert generated for Nmap (1)

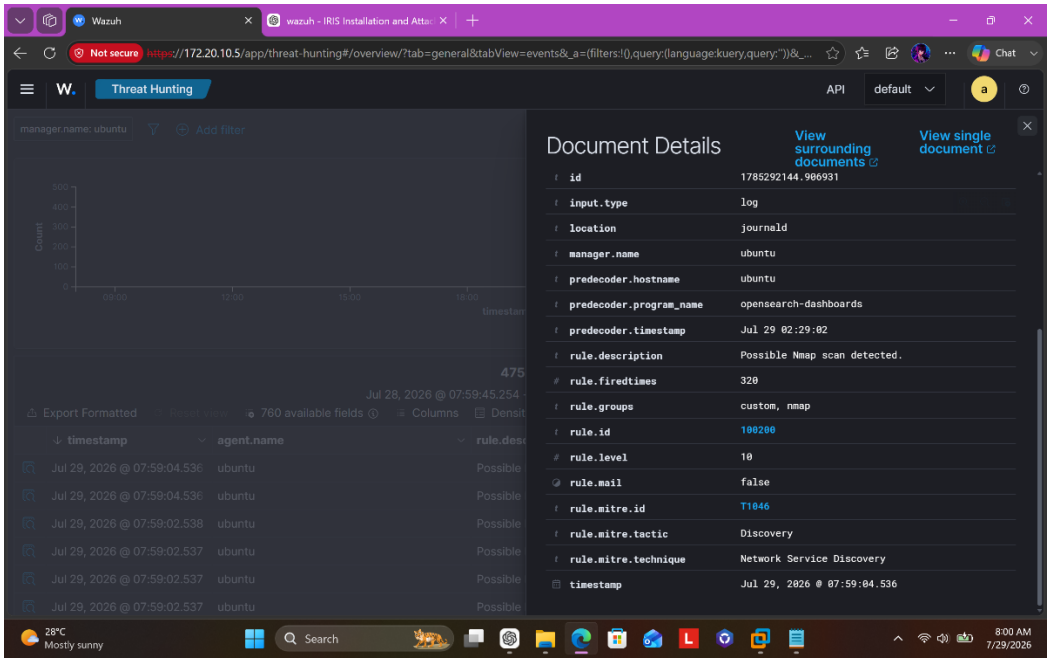


Figure 31: Alert generated for Nmap (2)

A wazuh alert is generate according to our custom rule and got classified into discovery tactic, but this scan was performed pre-compromise of a machine therefore, I am classifying this as a Recon activity.

2. Brute Forcing using Hydra (T1110.001)

2.1 Creating a Password Dictionary

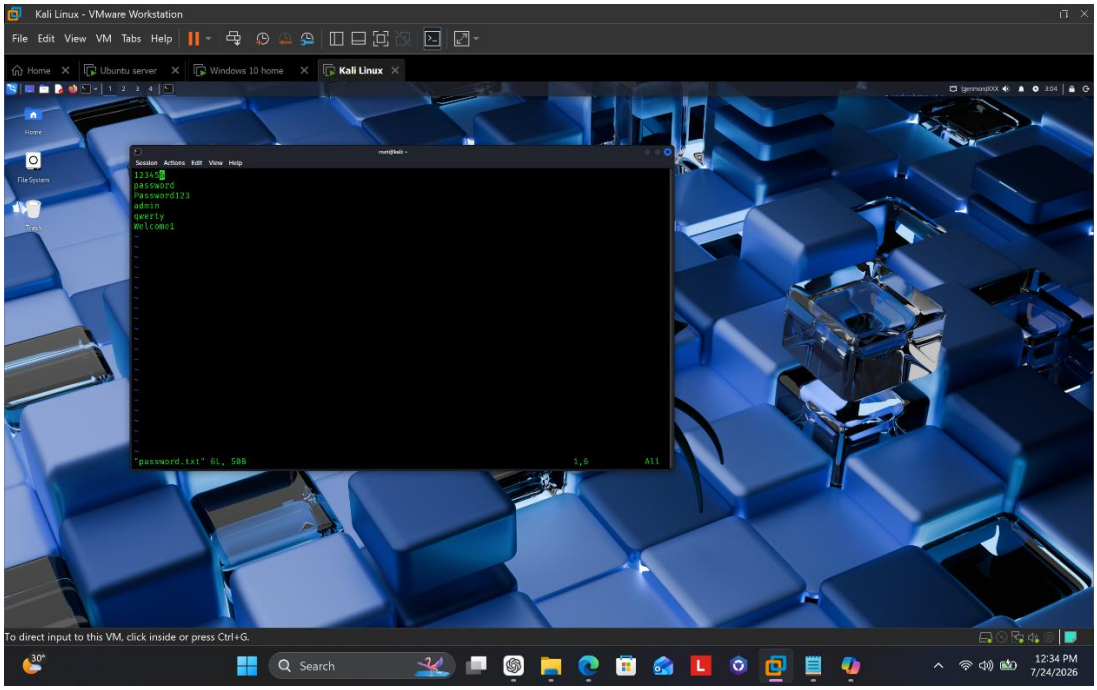


Figure 32: Creating a wordlst

2.2 Credential Guessing (Initial access) [TA0001]

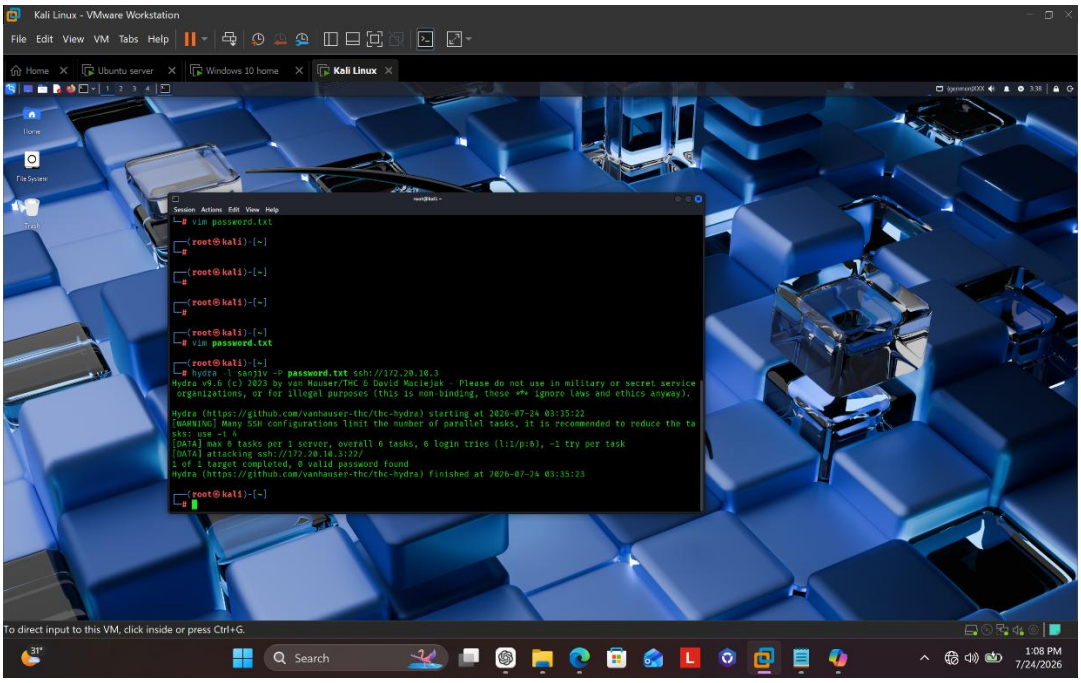


Figure 33: Brute Forcing using Hydra

2.3 Wazuh Detection

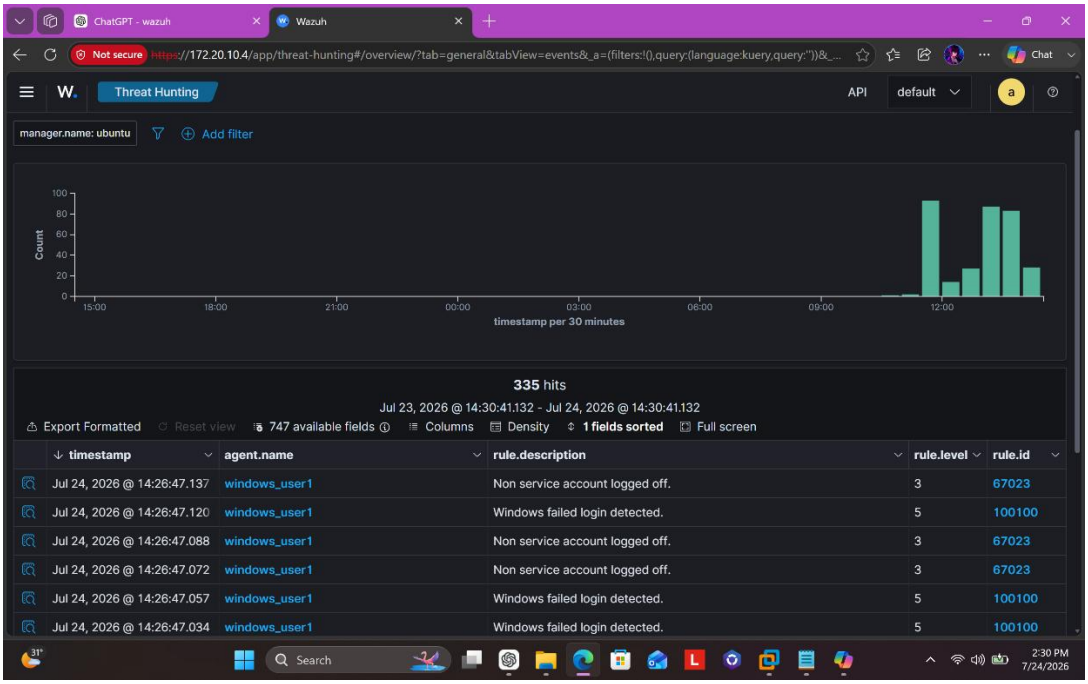


Figure 34: Alert generated for failed SSH

Overall 6 alerts were generated since my password dictionary contained 6 wordlist for network credential brute forcing using Hydra.

3. Establishing a PowerShell reverse shell (T1059.001)

3.1 C2 Listener Setup

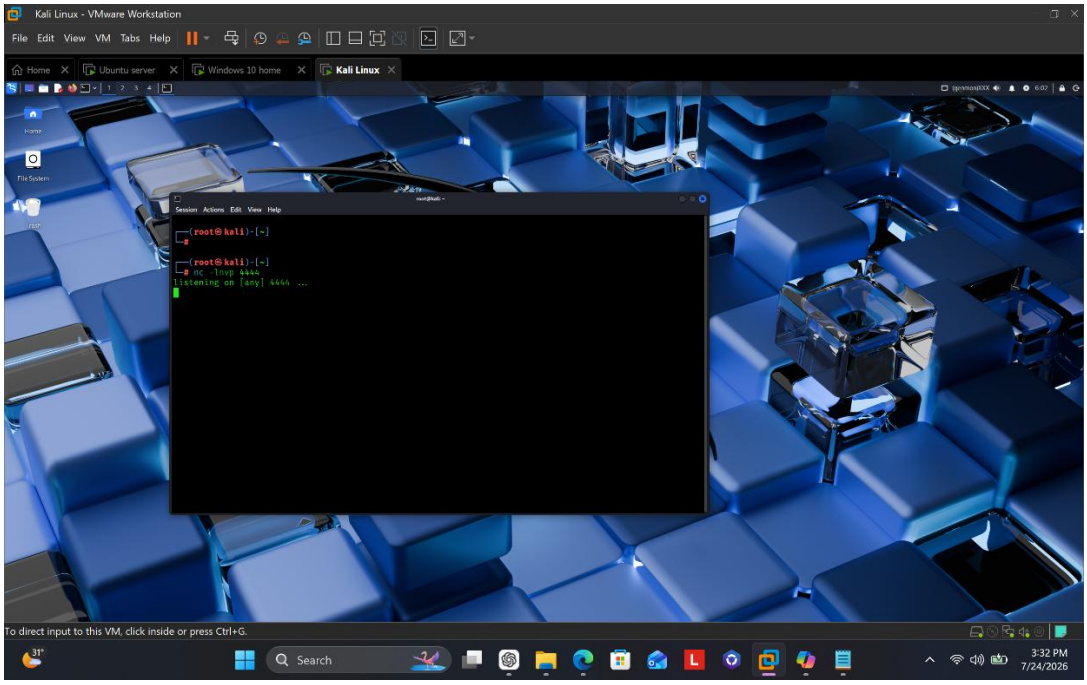


Figure 35: Establishing a listener for the reverse shell

3.2 Executing the PowerShell Reverse shell Payload

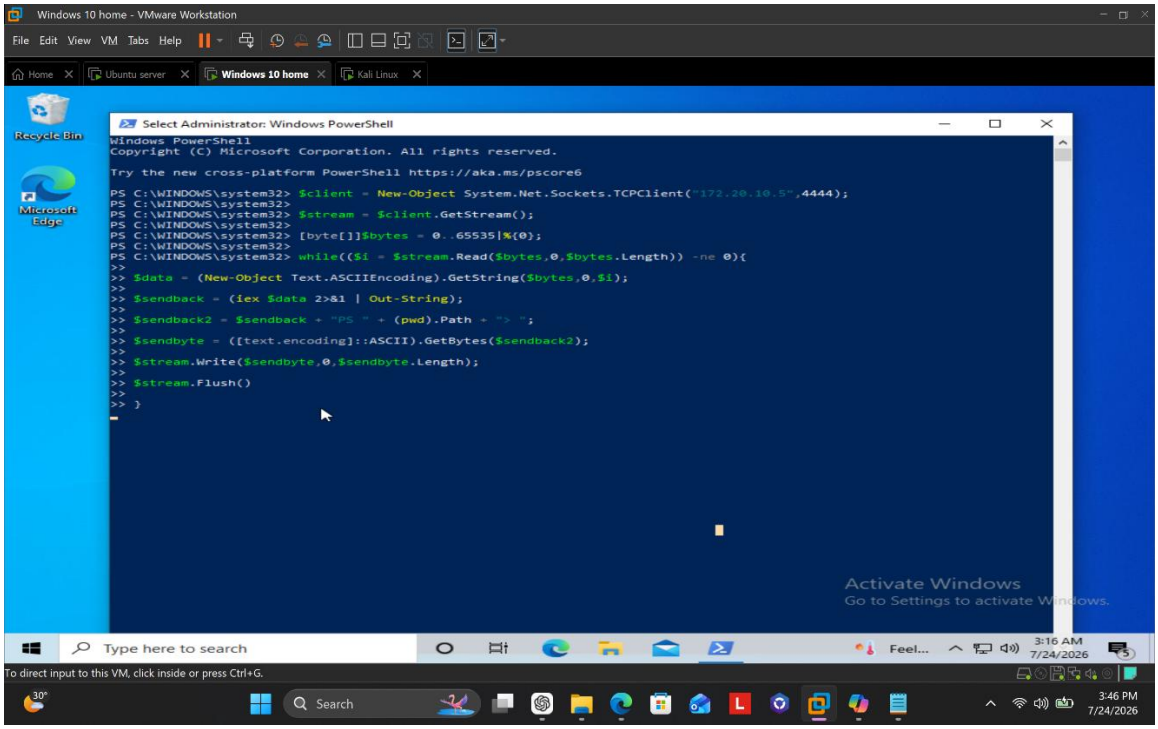


Figure 36: Executing the PowerShell based reverse TCP shell code in victim machine

3.3 Establishing C2(Command and Control) [TA0011]

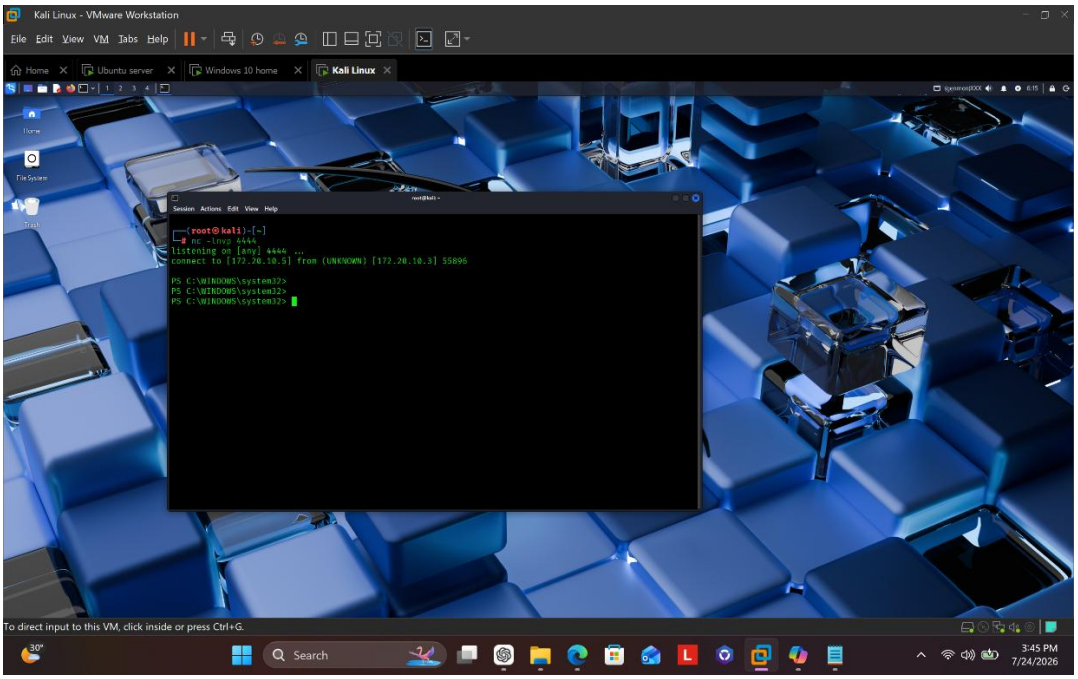


Figure 37: C2 Establishment

In my attacker machine I started a netcat listener on port 4444 and in the vicim machine I executed a powersell reverse shell payload which created a TCP connection to the attackers IP on port 4444, enabling remote command execution.

3.4 Wazuh Detection

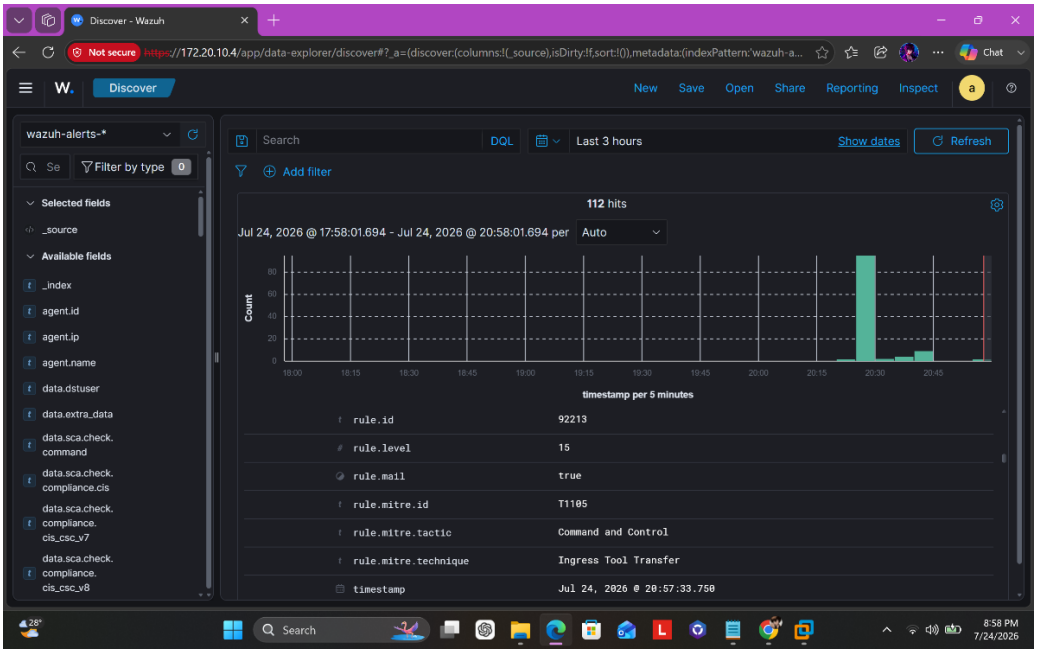


Figure 38: Alert generated for PowerShell Reverse shell

4. File Integrity Violation (T1565.001), (T1222.001)

4.1 Impact Stage of the Attack chain [TA0040]

C:\User1\Sensitive Data\employee_records.txt

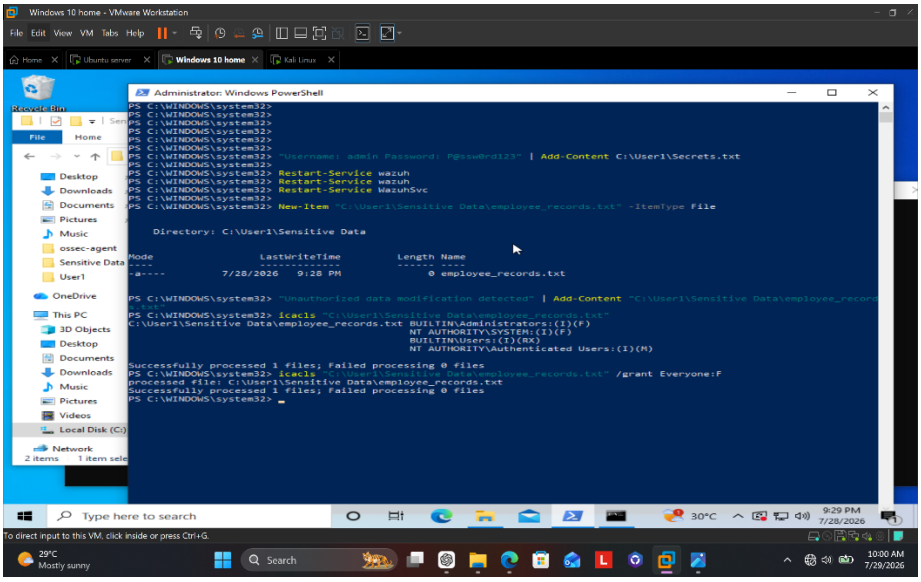


Figure 39: Stored data & permission modification

In this scenario I wanted to trigger a FIM alert in wazuh therefore, I executed a powershell command to create a new file, added content and changed file persmission to grant everyone full access over the file and then restarted the wazuh service to make sure the changes are recorded incase it wasn't.

4.2 Wazuh Detection

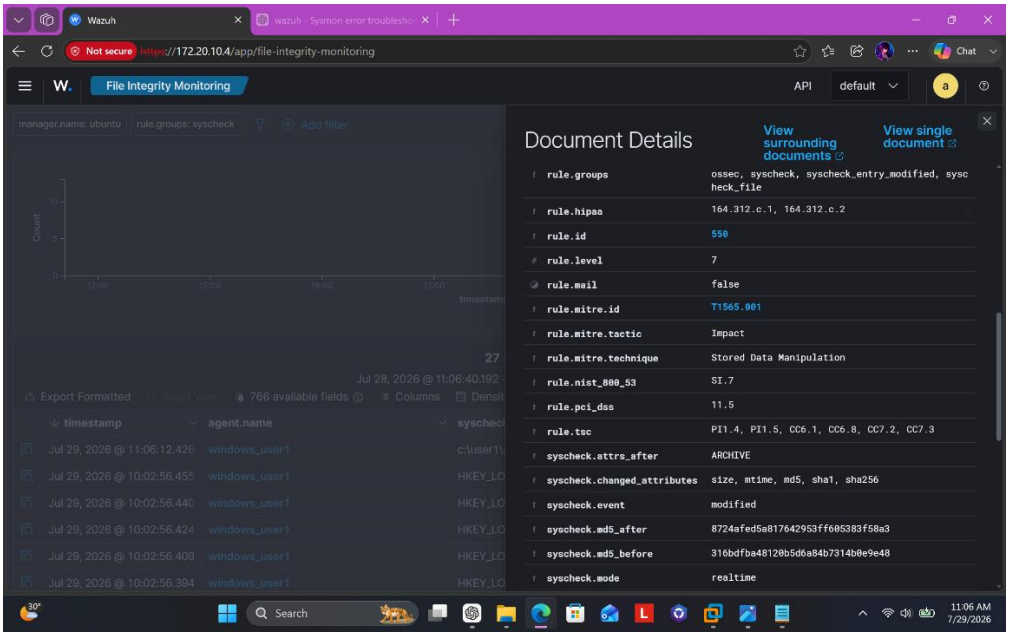


Figure 40: Alert generated for stored data manipulation

Iris Case Creation

1. Case Creation for Hydra Brute Force

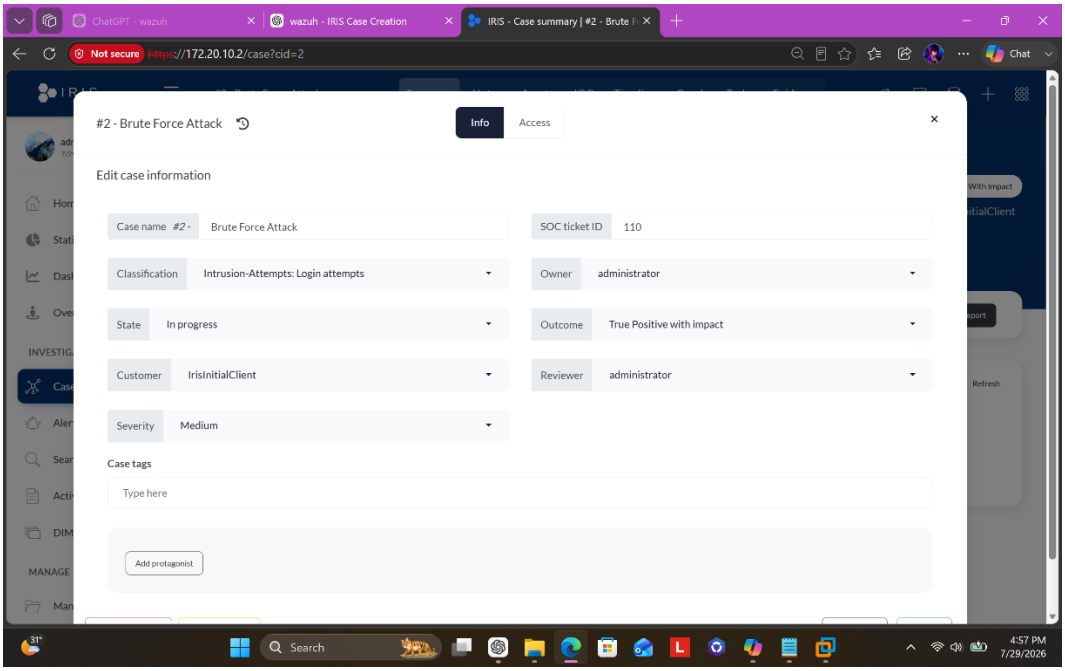


Figure 41: Creating a case for the Brute Force attack

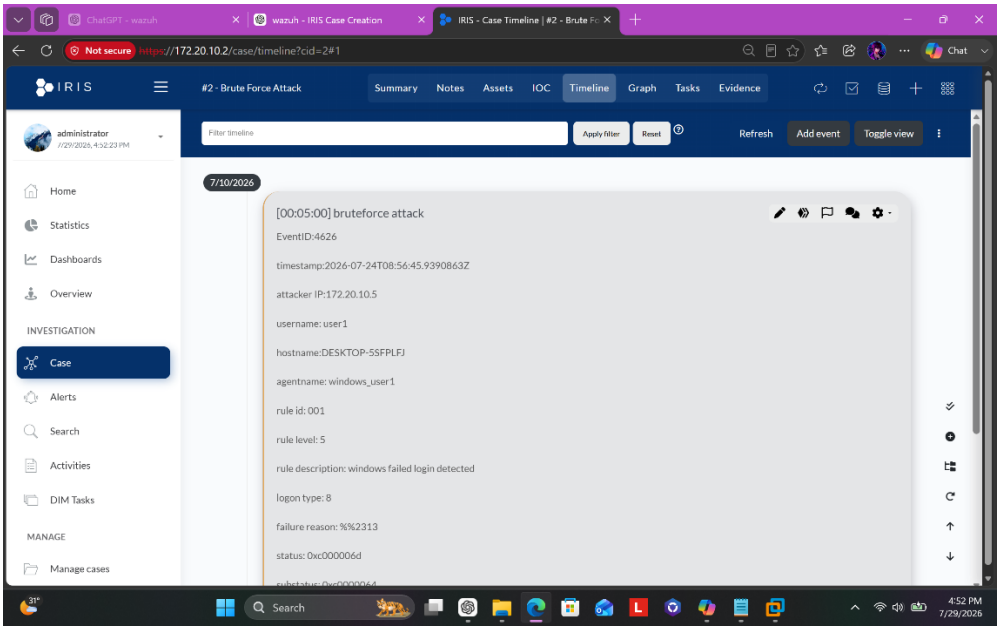


Figure 42: Adding the extracted information from wazuh

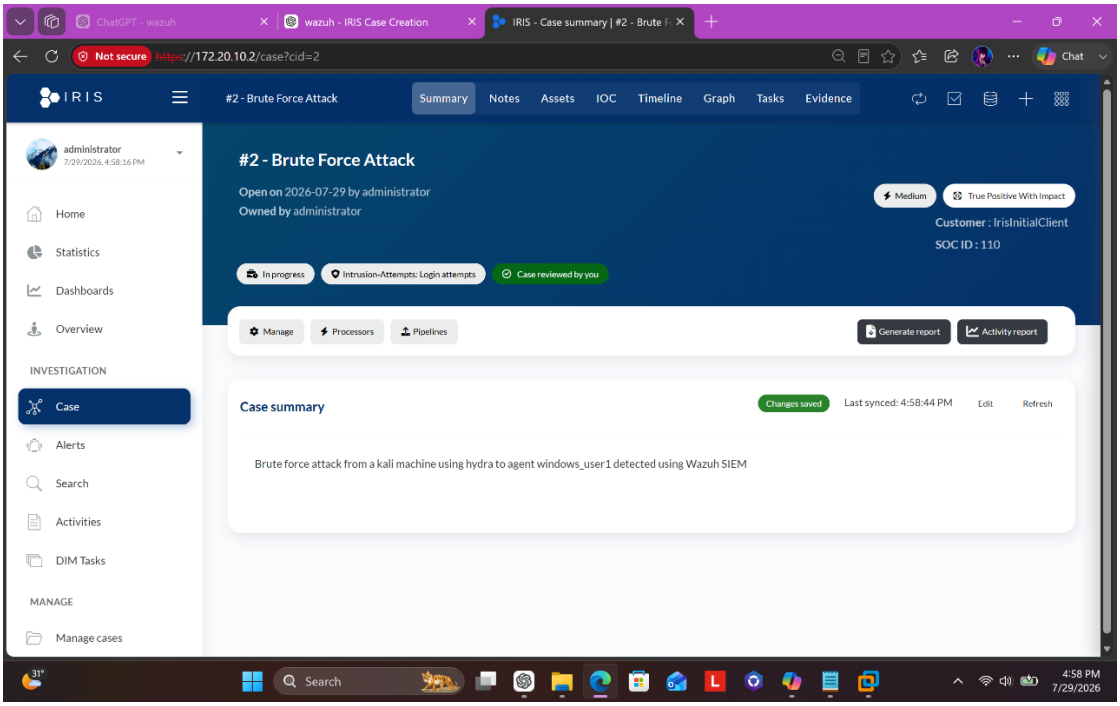


Figure 43: Successfully created a IRIS case for the Brute F orce attack

2. Case Creation for PowerShell Reverse shell

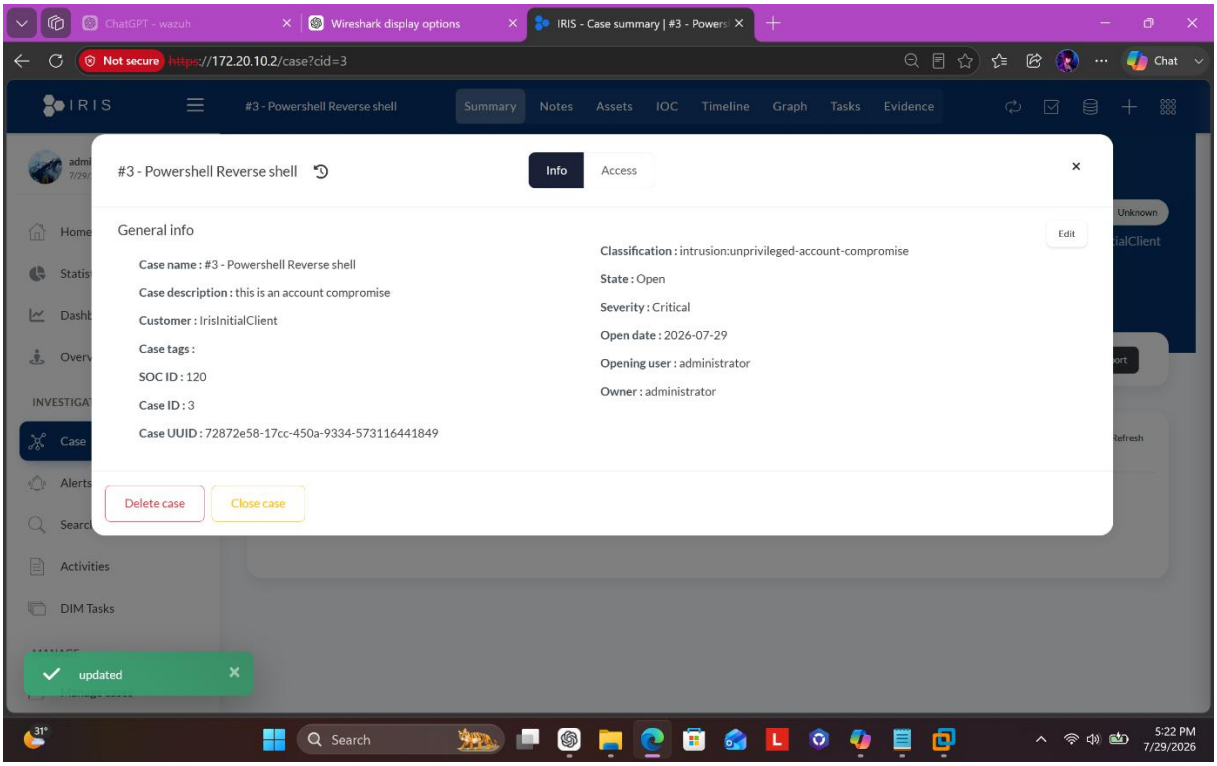


Figure 44: Creating a case for powershell reverse shell

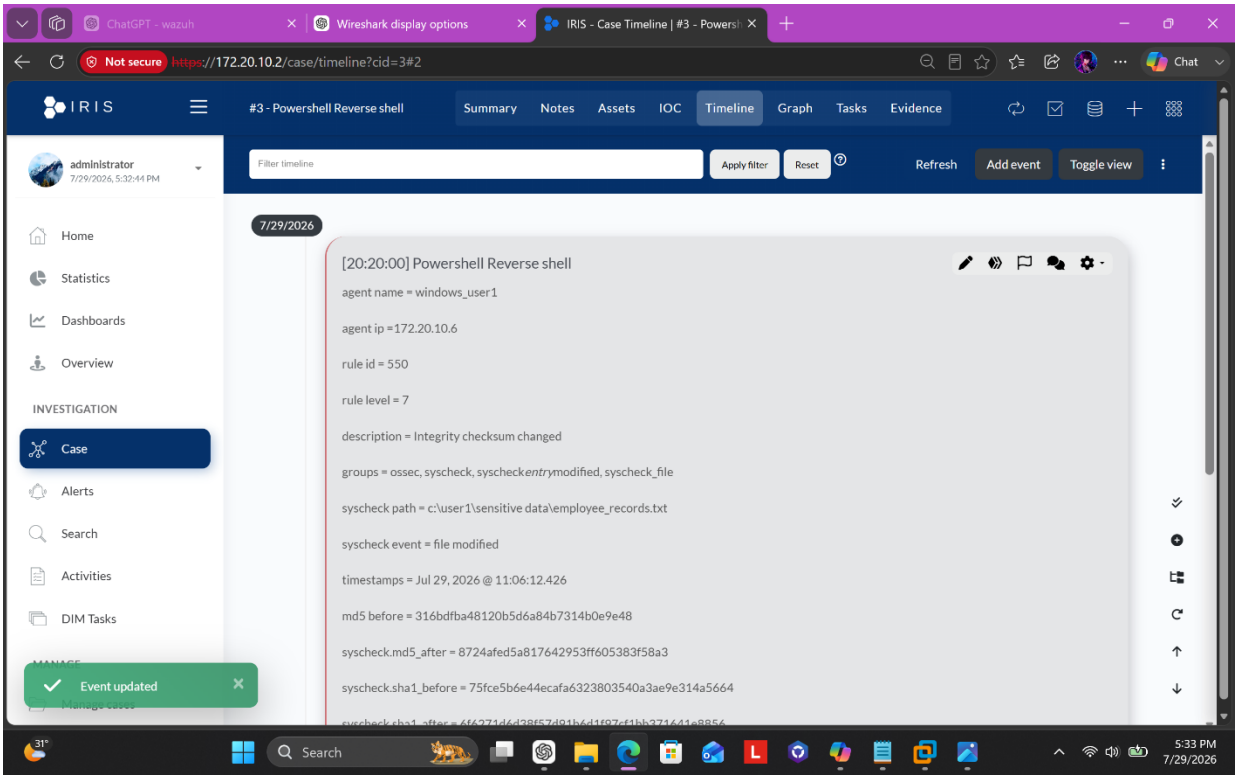


Figure 45: Adding the extracted information from wazuh

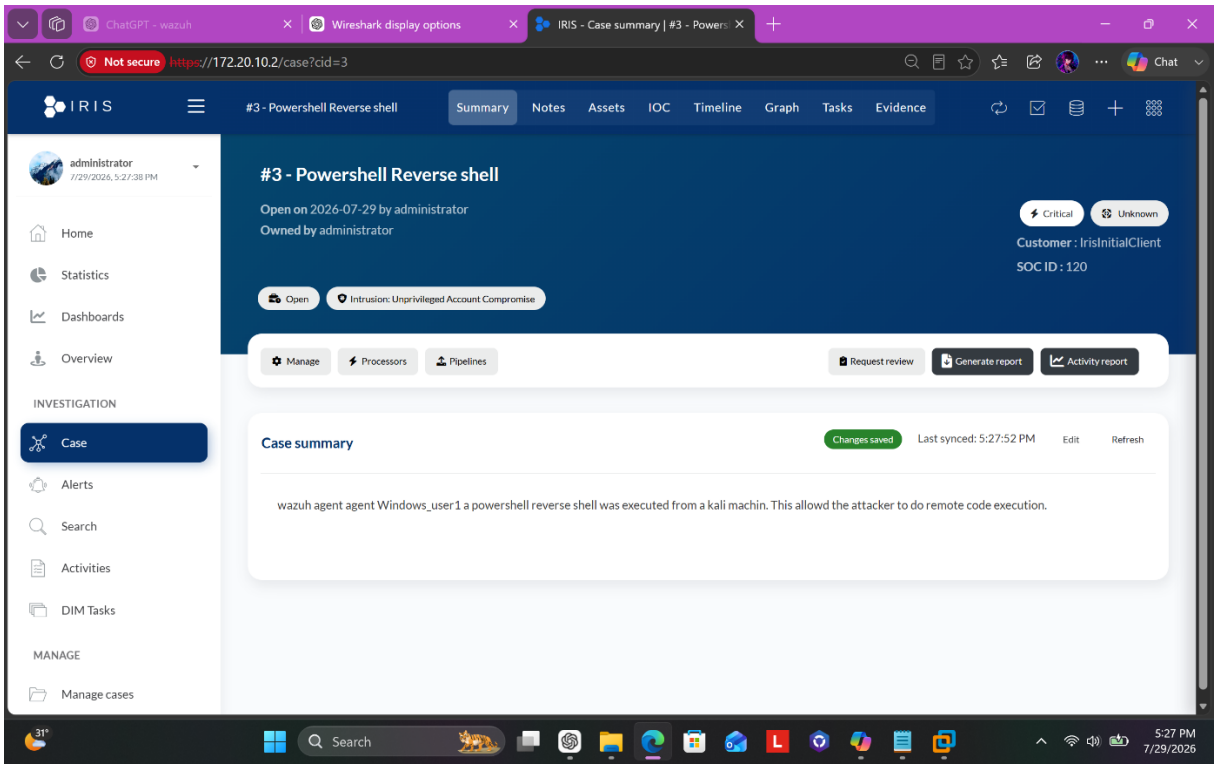


Figure 46: Successfully created a IRIS case for the powershell reverse shell

3. Case Creation for File Integrity Violation

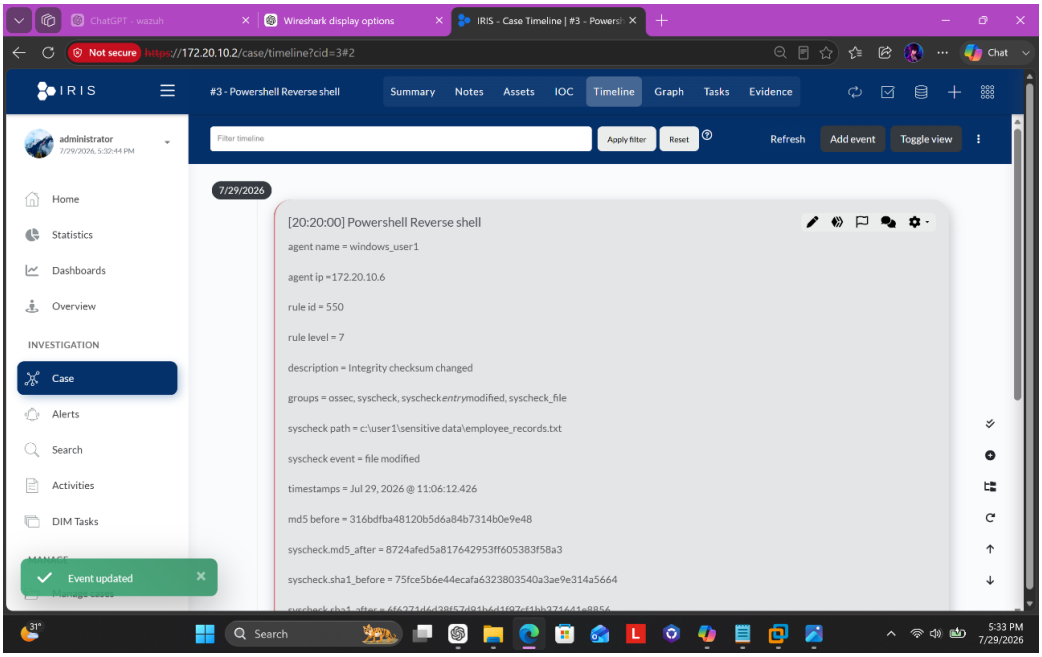


Figure 47: Adding the extracted information from wazuh

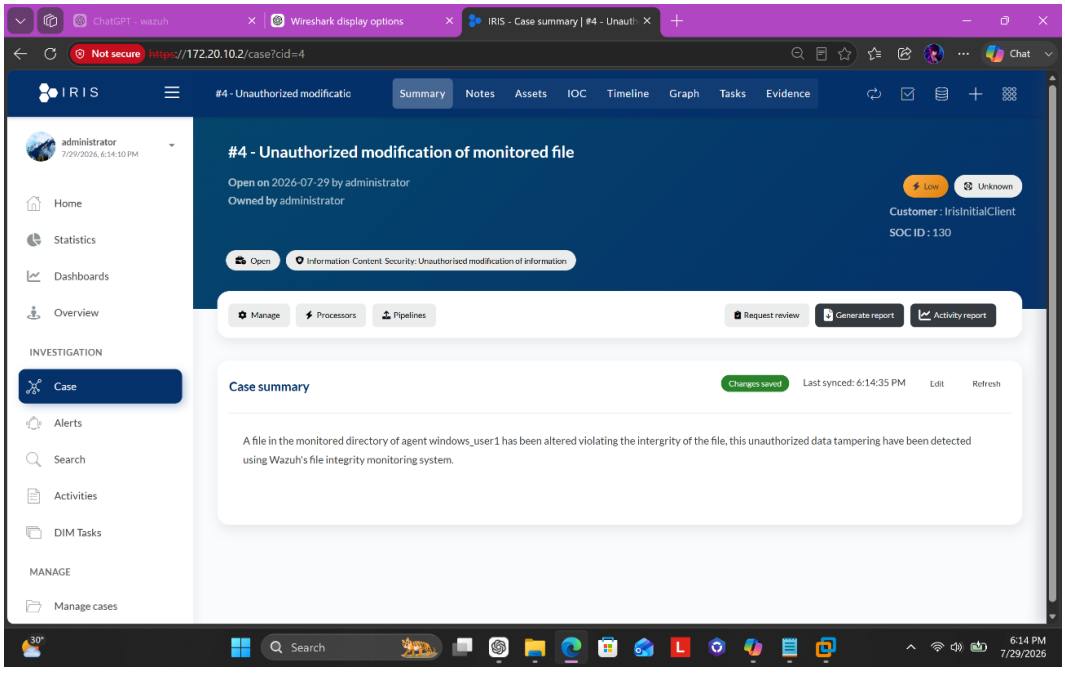


Figure 48: Successfully created a IRIS case for file integrity violation

- NOTE: No case was created for Nmap scan because attacker haven't compromised the machine or didn't attempt to yet, it was just a discovery of hosts, ports and services.

Conclusion

This project helped me gain hands on experience on configuring and implementing industry level tools like wazuh, security monitoring and threat detection platform and IRIS, digital forensics and incident response platform. I also understood the shortcomings of wazuh and how they are resolved like how logs are overwhelming if not refined or saturated using custom detection rules and it also re-triggered my awareness that wazuh does not directly inspect packets instead collect and analyzes network related events utilizing Sysmon to record the activity to windows event log entries and then the wazuh agent reads the windows event log sending the network events to wazuh manager. I also gained fundamental understanding of how attacks like Nmap scan, password guessing using hydra, remote code execution using PowerShell and stored data manipulation are executed and how they are mapped to MITRE ATT&CK framework making it possible to predict attacker behavior and classify alerts for better communication among the security team.